

Algorithm Design

Notes

Contents

1	Algorithm Analysis	5
1.1	Computational Tractability	5
1.1.1	Polynomial Running Time	5
1.1.2	Types of Analysis	6
1.2	Asymptotic Order of Growth	6
1.2.1	Big-O Notation (Upper Bound)	6
1.2.2	Big-Omega Notation (Lower Bound)	7
1.2.3	Big-Theta Notation (Tight Bound)	7
1.2.4	Useful Limit Rules	7
1.2.5	Key Asymptotic Facts	7
1.2.6	Big-O with Multiple Variables	8
1.3	Survey of Common Running Times	8
1.3.1	Logarithmic Time — $O(\log n)$	8
1.3.2	Linear Time — $O(n)$	8
1.3.3	Linearithmic Time — $O(n \log n)$	9
1.3.4	Quadratic Time — Quadratic Complexity	9
1.3.5	Cubic Time — Cubic Complexity	9
1.3.6	Polynomial Time — Polynomial Complexity	10
1.3.7	Exponential Time — Exponential Complexity	10
1.4	Hierarchy of Complexities	10
2	Greedy algorithm	12
2.1	Interval Scheduling	12
2.1.1	The Problem	12
2.1.2	Greedy Templates	12
2.1.3	Earliest-Finish-Time-First Algorithm	13
2.1.4	Proof of Optimality	13
2.2	Interval Partitioning	14
2.2.1	The Problem	14
2.2.2	Depth and Lower Bound	14
2.2.3	Earliest-Start-Time-First Algorithm	15
2.2.4	Proof of Optimality	15
2.3	Scheduling to Minimize Lateness	16
2.3.1	The Problem	16
2.3.2	Greedy Candidates	16
2.3.3	Earliest-Deadline-First Algorithm	17
2.3.4	Key Observations	17

2.3.5	Exchange Argument	17
2.3.6	Proof of Optimality	18
2.4	Greedy Analysis Strategies	18
3	Dynamic Programming	19
3.1	Algorithmic Paradigms	19
3.2	Weighted Interval Scheduling	19
3.2.1	Problem Definition	19
3.2.2	Key Definitions	20
3.2.3	Optimal Substructure (Binary Choice)	20
3.2.4	Brute Force	21
3.2.5	Memoization: Top-Down DP	21
3.2.6	Finding the Actual Solution	22
3.2.7	Bottom-Up DP	23
3.3	Segmented Least Squares	23
3.3.1	Background: Least Squares	23
3.3.2	Problem Definition	23
3.3.3	Optimal Substructure (Multiway Choice)	24
3.3.4	Algorithm	25
3.4	Knapsack Problem	25
3.4.1	Problem Definition	25
3.4.2	Why One Variable Is Not Enough	26
3.4.3	Optimal Substructure (New Variable)	26
3.4.4	Bottom-Up Algorithm	26
3.4.5	Running Time and Remarks	27
3.5	Sequence Alignment	27
3.5.1	String Similarity and Edit Distance	27
3.5.2	Problem Definition	28
3.5.3	Optimal Substructure	29
3.5.4	Algorithm	29
3.6	Hirschberg's Algorithm	30
3.6.1	Goal: Linear Space with Full Alignment	30
3.6.2	Divide-and-Conquer	31
3.6.3	Running Time Analysis	32
3.7	Bellman-Ford Algorithm	33
3.7.1	The Shortest Path Problem with Negative Weights	33
3.7.2	Negative Cycles	33
3.7.3	Dynamic Programming Formulation	34
3.7.4	Implementation	34
3.7.5	Bellman-Ford: Efficient Implementation	35
3.7.6	Analysis and Correctness	35
3.8	Distance Vector Protocols	36
3.8.1	Routing in Communication Networks	36
3.8.2	Distance Vector Protocols	36
3.8.3	Counting-to-Infinity Problem	36
3.8.4	Path Vector Protocols	37
3.9	Negative Cycle Detection	37
3.9.1	The Problem	37

3.9.2	Detection via Bellman-Ford	37
3.9.3	Algorithm	37
3.9.4	Summary of Complexities	38
4	exercises on dynamic programming	39
5	Network Flow	44
5.1	Max-Flow and Min-Cut Problems	44
5.2	Ford-Fulkerson Algorithm	45
5.2.1	Greedy Approach and Its Limits	45
5.2.2	Residual Graph	46
5.2.3	Augmenting Path	47
5.2.4	Ford-Fulkerson Algorithm	48
5.3	Max-Flow Min-Cut Theorem	49
5.3.1	Flow Value Lemma	49
5.3.2	Weak Duality	50
5.4	Max-Flow Min-Cut Theorem	50
5.4.1	Augmenting Path Theorem	51
5.5	Capacity-Scaling Algorithm	52
5.6	Shortest Augmenting Paths	53
5.6.1	Level Graph	53
5.7	Unit-Capacity Simple Networks	55
5.8	Bipartite Matching	55
5.8.1	Reduction to Max Flow	56
5.8.2	Hall's Theorem	56
5.8.3	Edge-Disjoint Paths	57
5.8.4	Reduction to Max Flow	57
5.8.5	Circulation with Demands	58
5.8.6	Lower Bounds	59
5.9	Survey Design	60
5.10	Airline Scheduling	60
5.10.1	Project Selection	60
6	exercises on Network Flow	62
7	NP and Computational Intractability	66
7.1	Algorithm Design Patterns and Anti-Patterns	66
7.2	Polynomial-Time Reductions (Section 8.1)	66
7.2.1	Classifying Problems	66
7.2.2	Polynomial-Time Reduction	67
7.2.3	Three Basic Reduction Strategies	67
7.2.4	Strategy 1 — Simple Equivalence	68
7.2.5	Strategy 2 — Special Case to General Case	68
7.2.6	Transitivity of Reductions	70
7.3	Definition of NP (Section 8.3)	70
7.3.1	Decision Problems	70
7.3.2	The Class P	71
7.3.3	The Class NP	71
7.3.4	P, NP, and EXP	72

7.3.5	The P vs. NP Question	72
7.4	NP-Completeness (Section 8.4)	73
7.4.1	Two Types of Reduction	73
7.4.2	NP-Complete Problems	73
7.4.3	Circuit Satisfiability — The First NP-Complete Problem	73
7.4.4	Recipe to Prove NP-Completeness	74
7.4.5	3-SAT is NP-Complete	75
7.4.6	The Domino Chain of NP-Complete Problems	75
8	APPROXIMATION ALGORITHMS(11)	76
8.1	Coping with NP-completeness	76
8.2	Load Balancing	77
8.2.1	The Problem	77
8.2.2	Algorithm 1: List Scheduling (Greedy)	77
8.2.3	Lower Bounds on the Optimal L^*	78
8.2.4	Analysis: List Scheduling is a 2-Approximation	78
8.2.5	Algorithm 2: Longest Processing Time (LPT)	79
8.3	Center Selection	80
8.3.1	The Problem	80
8.3.2	Wrong Greedy: A False Start	80
8.3.3	Correct Greedy Algorithm	80
8.4	Pricing Method: Vertex Cover	82
8.4.1	Weighted Vertex Cover	82
8.4.2	The Pricing Idea	82
8.4.3	The Pricing Algorithm	83
8.5	Coping with NP-completeness	84
8.6	Load Balancing (Section 11.1)	84
8.6.1	The Problem	84
8.6.2	Greedy Algorithm: List Scheduling	85
8.6.3	Analysis of List Scheduling	85
8.6.4	Longest Processing Time (LPT) Rule	85
8.7	Center Selection (Section 11.2)	86
8.7.1	The Problem	86
8.7.2	Greedy Algorithm: A False Start	86
8.7.3	The Correct Greedy Algorithm	86
8.8	Pricing Method: Vertex Cover (Section 11.4)	87
8.8.1	Weighted Vertex Cover	87
8.8.2	The Pricing Method Concept	87
8.8.3	Pricing Method Algorithm	87

Chapter 1

Algorithm Analysis

1.1 Computational Tractability

The question of how to solve a problem in the *shortest possible time* is not new. A simple approach to many problems is **brute force**: try every possible solution and pick the best one. The problem is that this usually takes 2^n time or worse, where n is the size of the input. For large inputs, this is completely impractical.

1.1.1 Polynomial Running Time

We say that an algorithm is **efficient** if its running time is *polynomial*.

Definition 1 (Polynomial-time algorithm). *An algorithm runs in polynomial time if there exist constants $c > 0$ and $d > 0$ such that, on every input of size n , its running time is at most $c \cdot n^d$ steps.*

A useful way to think about this: if you *double* the input size, the running time should only grow by a constant factor $C = 2^d$.

(To represent the number x we need $\log_2 x$ bits, so the input size is $n = \log_2 x$. If the algorithm runs in polynomial time, then the running time is $O(n^d) = O((\log_2 x)^d) = O(\log_2^d x)$)

The input change for combinatorial problems and numerical problems is different. For combinatorial problems, the input size is typically the number of elements (e.g., nodes in a graph), while for numerical problems, the input size is the number of bits needed to represent the numbers.

Why polynomial = efficient? In practice, poly-time algorithms tend to have small constants and small exponents. Also, breaking the exponential barrier usually means discovering an important structure in the problem.

Exceptions. A few poly-time algorithms have very large constants and are not useful in practice. On the other hand, some exponential-time algorithms (like the Simplex method) are widely used because the worst cases almost never happen in real life.

1.1.2 Types of Analysis

There are several ways to measure the running time of an algorithm:

Type	Description	Example
Worst-case	Guarantee for <i>any</i> input of size n	Heapsort: $\leq 2n \log_2 n$ compares
Probabilistic	Expected time of a randomised algorithm	Quicksort: $\sim 2n \ln n$ compares
Amortised	Worst-case over a sequence of n operations	Stack resize: $O(n)$ for n push/pop
Average-case	Expected time on a random input	3-way radix sort: $\sim 2n \ln n$

Worst-case analysis is the most common. It gives a solid guarantee for all possible inputs and usually reflects real performance well.

Sometimes the algorithms aren't deterministic, so we use **probabilistic analysis** to measure the expected running time. This is common for randomised algorithms, which use random bits to make decisions. In this case, we use the expected value of the running time over the random choices of the algorithm.

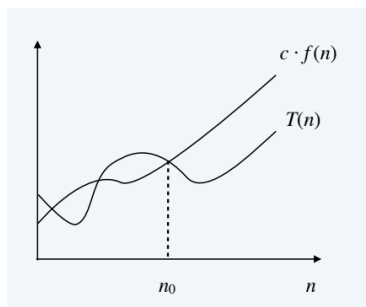
1.2 Asymptotic Order of Growth

Asymptotic notation lets us describe how the running time *grows* as n gets large, ignoring small constant factors.

1.2.1 Big-O Notation (Upper Bound)

Definition 2 (Big-O). $T(n)$ is $O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that

$$T(n) \leq c \cdot f(n) \quad \text{for all } n \geq n_0.$$



An equivalent condition using limits:

$$\limsup_{n \rightarrow \infty} \frac{T(n)}{f(n)} < \infty.$$

Example. Let $T(n) = 32n^2 + 17n + 1$. Then:

- $T(n)$ is $O(n^2)$ (choose $c = 50$, $n_0 = 1$)
- $T(n)$ is also $O(n^3)$
- $T(n)$ is *not* $O(n)$ nor $O(n \log n)$

Typical use. “Insertion sort makes $O(n^2)$ compares to sort n elements.”

Note on notation. Technically, $O(f(n))$ is a *set* of functions, but computer scientists write $T(n) = O(f(n))$ as shorthand. This is an accepted abuse of notation — just be careful not to misuse it (e.g., writing $O(n) = O(n^2)$ is fine, but not the reverse).

1.2.2 Big-Omega Notation (Lower Bound)

Definition 3 (Big-Omega). $T(n)$ is $\Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that

$$T(n) \geq c \cdot f(n) \quad \text{for all } n \geq n_0.$$

Example. For $T(n) = 32n^2 + 17n + 1$:

- $T(n)$ is $\Omega(n^2)$ and $\Omega(n)$
- $T(n)$ is not $\Omega(n^3)$

Typical use. “Any comparison-based sorting algorithm requires $\Omega(n \log n)$ compares in the worst case.”

Common mistake. It is meaningless to say “requires *at least* $O(n \log n)$ compares” — Big-O is an upper bound, not a lower bound.

1.2.3 Big-Theta Notation (Tight Bound)

Definition 4 (Big-Theta). $T(n)$ is $\Theta(f(n))$ if there exist constants $c_1 > 0$, $c_2 > 0$, and $n_0 \geq 0$ such that

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n) \quad \text{for all } n \geq n_0.$$

In other words, $T(n)$ is $\Theta(f(n))$ if and only if it is both $O(f(n))$ and $\Omega(f(n))$.

Example. Mergesort makes $\Theta(n \log n)$ compares. For $T(n) = 32n^2 + 17n + 1$: $T(n)$ is $\Theta(n^2)$, but not $\Theta(n)$ or $\Theta(n^3)$.

1.2.4 Useful Limit Rules

Proposition 1. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0$, then $f(n)$ is $\Theta(g(n))$.

Proposition 2. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, then $f(n)$ is $O(g(n))$ but not $\Omega(g(n))$.

Proof sketch for the first proposition. By definition of the limit, there exists n_0 such that for all $n \geq n_0$:

$$\frac{c}{2} < \frac{f(n)}{g(n)} < 2c \quad \Rightarrow \quad \frac{c}{2} \cdot g(n) \leq f(n) \leq 2c \cdot g(n).$$

This gives both the upper and lower bound needed for Θ . □

1.2.5 Key Asymptotic Facts

- **Polynomials.** If $T(n) = a_0 + a_1n + \dots + a_dn^d$ with $a_d > 0$, then $T(n)$ is $\Theta(n^d)$, because

$$\lim_{n \rightarrow \infty} \frac{a_0 + a_1n + \dots + a_dn^d}{n^d} = a_d > 0.$$

- **Logarithm base does not matter.** $\Theta(\log_a n) = \Theta(\log_b n)$ for any constants $a, b > 0$ (by change-of-base formula). We simply write $\log n$.

- **Logs are slower than polynomials.** For every $d > 0$, $\log n$ is $O(n^d)$.
- **Polynomials are slower than exponentials.** For every $r > 1$ and $d > 0$, n^d is $O(r^n)$, i.e.,

$$\lim_{n \rightarrow \infty} \frac{n^d}{r^n} = 0.$$

1.2.6 Big-O with Multiple Variables

Sometimes the input has two parameters (e.g., nodes and edges in a graph).

Definition 5. $T(m, n)$ is $O(f(m, n))$ if there exist constants $c > 0$, $m_0 \geq 0$, $n_0 \geq 0$ such that $T(m, n) \leq c \cdot f(m, n)$ for all $n \geq n_0$ and $m \geq m_0$.

Example. $T(m, n) = 32mn^2 + 17mn + 32n^3$ is $O(mn^2 + n^3)$ and also $O(mn^3)$, but it is neither $O(n^3)$ nor $O(mn^2)$.

Typical use. Breadth-first search (BFS) takes $O(m + n)$ time to find the shortest path from s to t in a directed graph with n nodes and m edges.

1.3 Survey of Common Running Times

The table below shows the most common complexity classes, from fastest to slowest.

Complexity	Name	Example
$O(\log n)$	Sublinear / Logarithmic	Binary search
$O(n)$	Linear	Finding max, merging two lists
$O(n \log n)$	Linearithmic	Mergesort, Heapsort
$O(n^2)$	Quadratic	Closest pair (naive)
$O(n^3)$	Cubic	Set disjointness
$O(n^k)$	Polynomial	Independent set of size k
$O(2^n)$	Exponential	Maximum independent set

1.3.1 Logarithmic Time — $O(\log n)$

The running time grows very slowly — doubling the input only adds one step.

Binary search. Given a sorted array A of n numbers, find whether x is in the array.

```

lo = 1, hi = n
while (lo <= hi):
    mid = (lo + hi) / 2
    if x < A[mid]: hi = mid - 1
    elif x > A[mid]: lo = mid + 1
    else: return YES
return NO

```

Each iteration cuts the search interval in half, so at most $\lceil \log_2 n \rceil$ iterations are needed.

1.3.2 Linear Time — $O(n)$

The running time grows proportionally to the input size.

Example 1 — Maximum of n numbers.

```
max = a[1]
for i = 2 to n:
    if a[i] > max:
        max = a[i]
```

Example 2 — Merging two sorted lists $A = a_1, \dots, a_n$ and $B = b_1, \dots, b_n$:

```
i = 1, j = 1
while (both lists are nonempty):
    if a[i] <= b[j]: append a[i] to output; i++
    else:           append b[j] to output; j++
append the rest of the nonempty list
```

Claim. Merging takes $O(n)$ time. After each compare, the output list grows by one element, so at most $2n - 1$ compares are needed.

1.3.3 Linearithmic Time — $O(n \log n)$

This complexity often appears in divide-and-conquer algorithms.

- **Sorting:** Mergesort and Heapsort both use $O(n \log n)$ compares.
- **Largest empty interval:** Given n timestamps $x_1, \dots, x_n$ when copies of a file arrive at a server, find the longest gap with no arrivals. *Solution:* sort the timestamps ($O(n \log n)$), then scan in order and track the maximum gap between consecutive timestamps ($O(n)$). Total: $O(n \log n)$.

1.3.4 Quadratic Time — Quadratic Complexity

Typical when you enumerate all *pairs* of elements.

Closest pair of points. Given n points $(x_1, y_1), \dots, (x_n, y_n)$ in the plane, find the closest pair.

```
min = dist(p1, p2)
for i = 1 to n:
    for j = i+1 to n:
        d = (xi - xj)^2 + (yi - yj)^2
        if d < min: min = d
```

This naive solution is $O(n^2)$. It might seem that $\Omega(n^2)$ is unavoidable, but in Chapter 5 (divide-and-conquer) we will see an $O(n \log n)$ solution.

1.3.5 Cubic Time — Cubic Complexity

Typical when you enumerate all *triples* of elements.

Set disjointness. Given n sets $S_1, \dots, S_n \subseteq \{1, \dots, n\}$, is there a disjoint pair?

```

for each set Si:
  for each set Sj (j != i):
    for each element p in Si:
      check if p is in Sj
    if no element of Si is in Sj:
      report Si and Sj disjoint

```

The three nested loops give $O(n^3)$ time.

1.3.6 Polynomial Time — Polynomial Complexity

Useful when k is a small constant, but may be impractical for large k .

Independent set of size k . Given a graph, are there k nodes with no edge between any two of them?

```

for each subset S of k nodes:
  if S is an independent set: report S

```

The number of k -element subsets is:

$$\binom{n}{k} = \frac{n(n-1) \cdots (n-k+1)}{k!} \leq \frac{n^k}{k!}.$$

Each check takes $O(k^2)$ time, so the total is $O(k^2 \cdot n^k/k!) = O(n^k)$. This is polynomial for any fixed k , but already impractical for $k = 17$.

1.3.7 Exponential Time — Exponential Complexity

Maximum independent set. Find the largest set of nodes with no edge between any two.

```

S* = empty
for each subset S of nodes:
  if S is independent and |S| > |S*|:
    S* = S

```

There are 2^n subsets, and each check is $O(n^2)$, so the total is $O(n^2 \cdot 2^n)$. This is only feasible for very small n .

1.4 Hierarchy of Complexities

The following chain summarises how the classes relate:

$$O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(n^k) \subset O(2^n).$$

Each class grows strictly faster than the one on its left. In particular:

- Every logarithm grows slower than every polynomial.

- Every polynomial grows slower than every exponential.

These facts follow directly from the limit rules in Section 2.4.

Chapter 2

Greedy algorithm

2.1 Interval Scheduling

2.1.1 The Problem

Input. A set of n jobs. Job j starts at time s_j and finishes at time f_j .

Compatibility. Two jobs are *compatible* if they don't overlap in time.

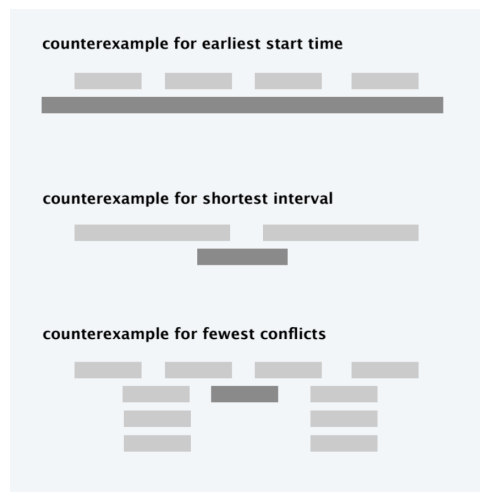
Goal. Find the maximum number of mutually compatible jobs.

2.1.2 Greedy Templates

Several natural greedy rules can be tried:

1. **Earliest start time first:** schedule jobs in ascending order of s_j .
2. **Earliest finish time first:** schedule jobs in ascending order of f_j .
3. **Shortest interval first:** schedule jobs in ascending order of $f_j - s_j$.
4. **Fewest conflicts first:** for each job j , count the number of conflicting jobs c_j , then schedule in ascending order of c_j .

Counterexamples show that only earliest finish time works.



2.1.3 Earliest-Finish-Time-First Algorithm

EARLIEST-FINISH-TIME-FIRST ($n, s_1, s_2, \dots, s_n, f_1, f_2, \dots, f_n$)

SORT jobs by finish time so that $f_1 \leq f_2 \leq \dots \leq f_n$

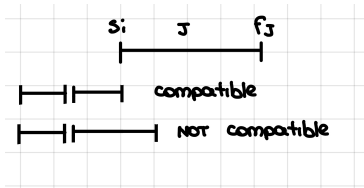
$A \leftarrow \phi$ ← set of jobs selected

FOR $j = 1$ **TO** n

IF job j is compatible with A

$A \leftarrow A \cup \{j\}$

RETURN A

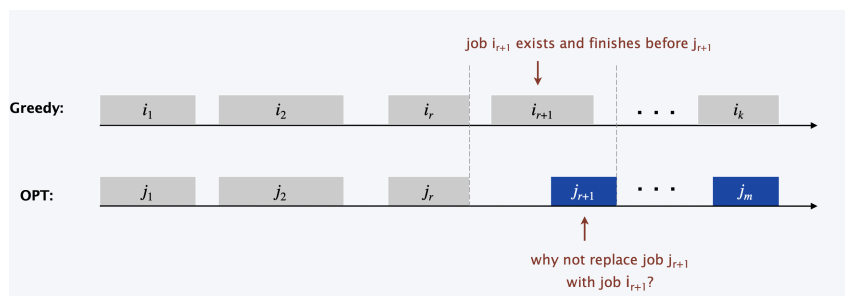


Implementation note. Keep track of the last job j^* added to A . Job j is compatible with A if and only if $s_j \geq f_{j^*}$. Sorting takes $O(n \log n)$ time; the loop is $O(n)$. Total: $O(n \log n)$.

2.1.4 Proof of Optimality

Theorem 1. *The earliest-finish-time-first algorithm is optimal.*

Proof (by contradiction). Assume the greedy algorithm is not optimal. Let $i_1, i_2, \dots, i_k$ be the jobs selected by the greedy algorithm, and let $j_1, j_2, \dots, j_m$ be the jobs in an optimal solution with $i_1 = j_1, i_2 = j_2, \dots, i_r = j_r$ for the largest possible r . (At r , the greedy and optimal solutions first differ.) If $r = k$, then the greedy solution has k jobs and the optimal has $m \geq k$, so they are equal (both optimal). Otherwise, job i_{r+1} exists. Since the greedy algorithm picks jobs by earliest finish time, $f_{i_{r+1}} \leq f_{j_{r+1}}$. We can replace j_{r+1} with i_{r+1} in the optimal solution. This creates a new optimal solution with one more agreement, contradicting the maximality of r .



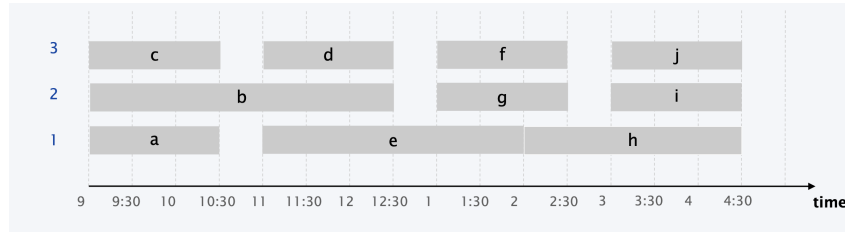
□

2.2 Interval Partitioning

2.2.1 The Problem

Input. A set of n lectures. Lecture j starts at s_j and finishes at f_j .

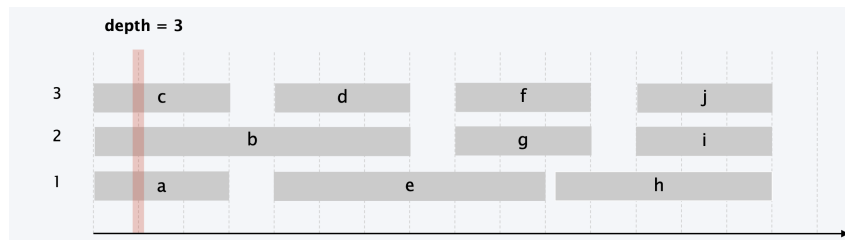
Goal. Schedule all lectures using the *minimum number* of classrooms so that no two lectures overlap in the same classroom.



2.2.2 Depth and Lower Bound

Definition 6 (Depth). The depth of a set of intervals is the maximum number of intervals that contain any given time.

When we talk about lower bound, we mean a number that is less than or equal to the optimal solution. Since the depth is a lower bound, it means that no schedule can use fewer classrooms than the depth.



Key observation. The number of classrooms needed is at least the depth. The optimal number always equal the depth and the earliest-start-time-first algorithm achieves it.

2.2.3 Earliest-Start-Time-First Algorithm

EARLIEST-START-TIME-FIRST ($n, s_1, s_2, \dots, s_n, f_1, f_2, \dots, f_n$)

Sort lectures by start time so that $s_1 \leq s_2 \leq \dots \leq s_n$.

$d \leftarrow 0$ ← number of allocated classrooms

FOR $j = 1$ **TO** n

IF lecture j is compatible with some classroom

 Schedule lecture j in any such classroom k .

ELSE

 Allocate a new classroom $d + 1$.

 Schedule lecture j in classroom $d + 1$.

$d \leftarrow d + 1$

RETURN schedule.

Implementation. Use a priority queue to store classrooms, keyed by the finish time of their last lecture. To check if lecture j is compatible, compare s_j to the minimum key in the queue. If compatible, assign j to that classroom and update its key to f_j .

Running time: Sorting takes $O(n \log n)$. Each of the n lectures is processed once, and each priority queue operation takes $O(\log n)$, so the total is $O(n \log n)$.

2.2.4 Proof of Optimality

Theorem 2. *The earliest-start-time-first algorithm is optimal.*

Proof. Let d be the number of classrooms the algorithm uses. Classroom d was opened because we had to schedule some lecture j that was incompatible with all $d - 1$ existing classrooms. This means there are d lectures that all overlap at time $s_j + \varepsilon$ (lecture j plus one from each of the other $d - 1$ classrooms). Since these d lectures overlap, any schedule must use at least d classrooms. The algorithm uses exactly d classrooms, so it is optimal. $\square$

2.3 Scheduling to Minimize Lateness

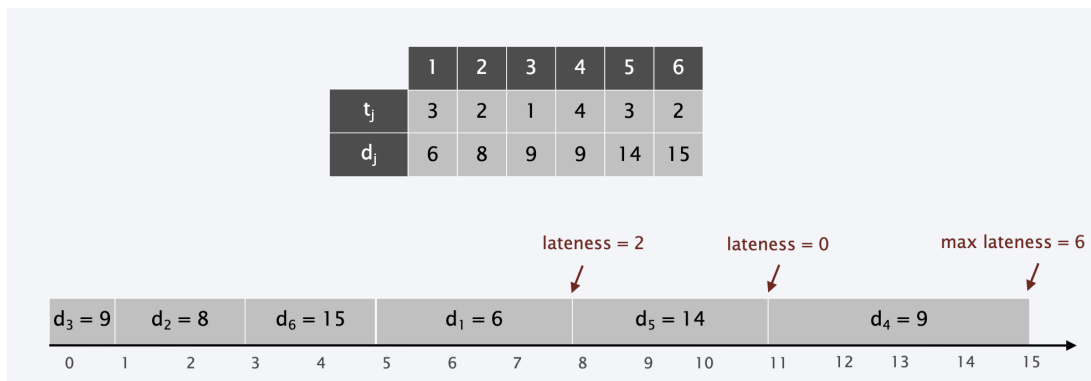
2.3.1 The Problem

Input. A single resource processes one job at a time. Job j requires t_j units of processing time and has a deadline d_j .

Definitions.

- If job j starts at time s_j , it finishes at time $f_j = s_j + t_j$.
- The *lateness* of job j is $\ell_j = \max\{0, f_j - d_j\}$.

Goal. Schedule all jobs to minimize the *maximum lateness* $L = \max_j \ell_j$.



2.3.2 Greedy Candidates

1. **Shortest processing time first:** schedule jobs in ascending order of t_j .
2. **Earliest deadline first:** schedule jobs in ascending order of d_j .
3. **Smallest slack:** schedule jobs in ascending order of $d_j - t_j$ (slack = deadline minus processing time).

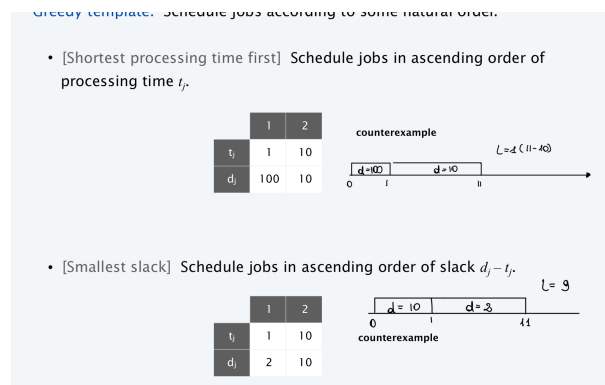
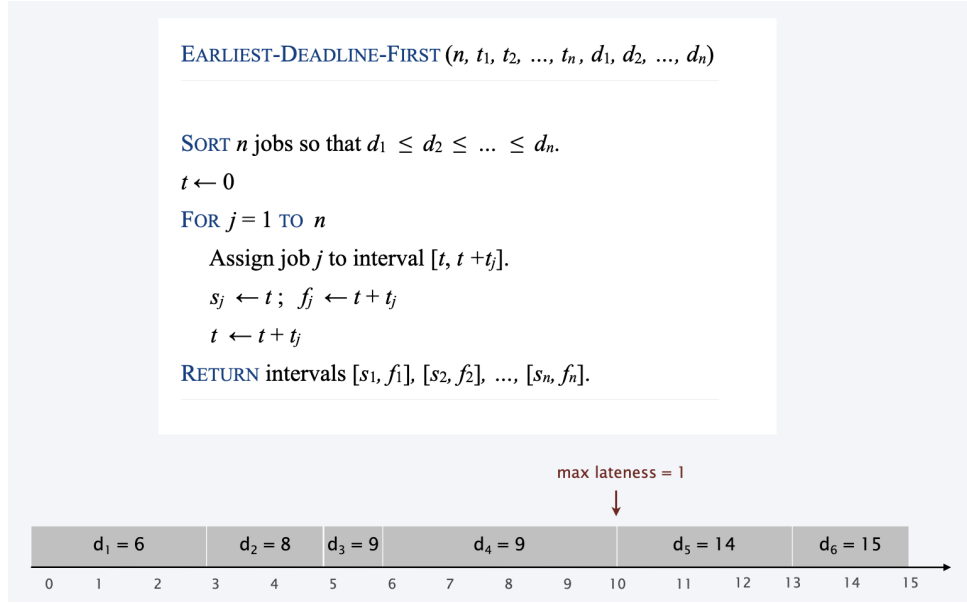


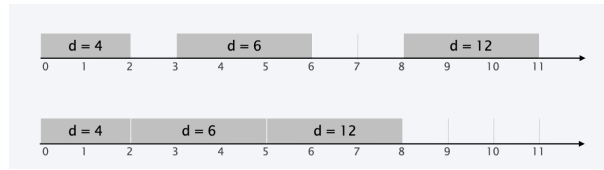
Figure 2.1: Counterexample showing that only earliest-deadline-first works.

2.3.3 Earliest-Deadline-First Algorithm



2.3.4 Key Observations

Observation 1. *There exists an optimal schedule with no idle time.*



Observation 2. *The earliest-deadline-first schedule has no idle time.*

Definition 7 (Inversion). *Given a schedule S (with jobs numbered so that $d_1 \leq d_2 \leq \dots \leq d_n$), an inversion is a pair of jobs i and j such that $i < j$ but j is scheduled before i .*

Observation 3. *The earliest-deadline-first schedule has no inversions.*

Observation 4. *If a schedule (with no idle time) has an inversion, it has one with a pair of inverted jobs scheduled consecutively.*

2.3.5 Exchange Argument

Claim 1. *Swapping two adjacent, **inverted** jobs reduces the number of inversions by one and does not increase the maximum lateness.*

Proof. Let ℓ be the lateness before the swap, and ℓ' after.

Let $i < j$ be the inverted pair (so $d_i \leq d_j$). For all $k \neq i, j$, we have $\ell'_k = \ell_k$. Job i now finishes earlier, so $\ell'_i \leq \ell_i$. For job j , if it is late:

$$\ell'_j = f'_j - d_j = f_i - d_j \leq f_i - d_i \leq \ell_i,$$

where the first inequality uses $d_i \leq d_j$ (since $i < j$). So the maximum lateness does not increase. $\square$

2.3.6 Proof of Optimality

Theorem 3. *The earliest-deadline-first schedule is optimal.*

Proof (by contradiction). Let S^* be an optimal schedule with the fewest number of inversions. We can assume S^* has no idle time. If S^* has no inversions, then $S^* = S_{\text{EDF}}$ and we are done. Otherwise, let $i-j$ be an adjacent inversion in S^* . Swapping i and j does not increase the maximum lateness and strictly decreases the number of inversions. This contradicts the assumption that S^* has the fewest inversions among optimal schedules. $\square$

2.4 Greedy Analysis Strategies

We have seen three main techniques for proving that a greedy algorithm is optimal:

1. **Greedy stays ahead.** Show that after each step, the greedy solution is at least as good as any other solution.

Example: Interval scheduling (earliest-finish-time-first).

2. **Structural bound.** Identify a simple lower bound that every solution must satisfy, then show that the greedy algorithm achieves this bound.

Example: Interval partitioning (depth = lower bound on number of classrooms).

3. **Exchange argument.** Gradually transform any optimal solution into the greedy solution by making small changes (“exchanges”) that do not worsen the objective.

Examples: Minimizing lateness (swapping inversions), Optimal caching (changing eviction decisions).

Other famous greedy algorithms: Gale–Shapley (stable matching), Kruskal and Prim (minimum spanning tree), Dijkstra (shortest paths), Huffman (data compression), and many more.

Chapter 3

Dynamic Programming

3.1 Algorithmic Paradigms

There are three main design strategies for algorithms:

1. **Greedy.** Build a solution step by step, always making the locally best choice at each step.
2. **Divide-and-conquer.** Split the problem into *independent* subproblems, solve each one, and combine the results.
3. **Dynamic programming.** Split the problem into *overlapping* subproblems, solve each one once, and store the result for later reuse.

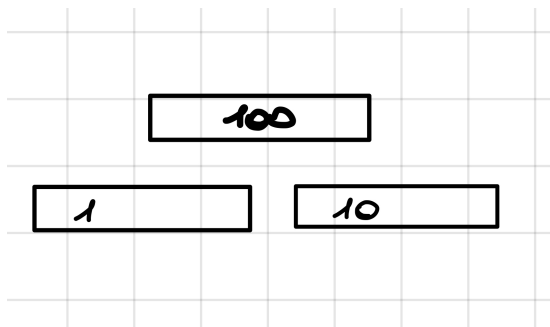
Dynamic programming is essentially a fancy name for *caching intermediate results in a table* so they do not have to be recomputed. The key difference from divide-and-conquer is that subproblems overlap: the same subproblem appears in many branches of the recursion.

3.2 Weighted Interval Scheduling

3.2.1 Problem Definition

Input. A set of n jobs. Job j starts at time s_j , finishes at f_j , and has value (weight) $v_j > 0$. Two jobs are *compatible* if they do not overlap.

Goal. Find a maximum-weight subset of mutually compatible jobs.



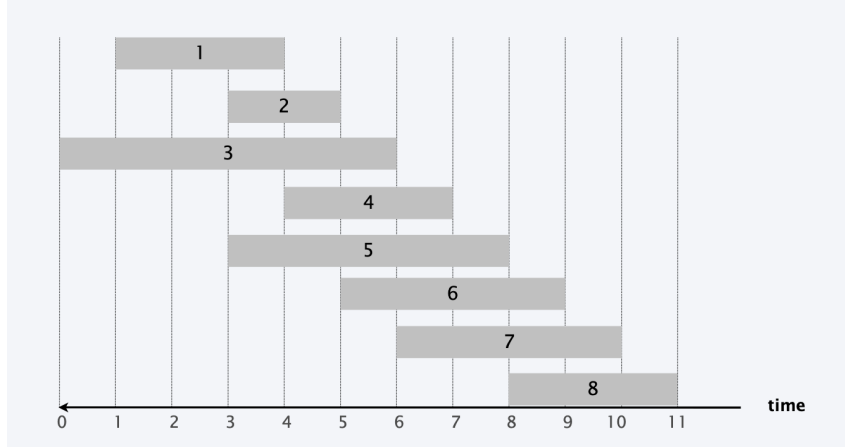
Why greedy fails. Counterexample showing that earliest-finish-time-first fails when jobs have weights because a single high-weight job (e.g. $v=100$) that overlaps with many low-weight jobs will be ignored by greedy, even though it is clearly the best choice.

3.2.2 Key Definitions

Label jobs by finishing time: $f_1 \leq f_2 \leq \dots \leq f_n$.

Definition 8 (Compatible predecessor $p(j)$). $p(j)$ is the largest index $i < j$ such that job i is compatible with job j (i.e., $f_i \leq s_j$). If no such job exists, $p(j) = 0$.

Example. For a set of 8 jobs: $p(8) = 5$, $p(7) = 3$, $p(2) = 0$.



3.2.3 Optimal Substructure (Binary Choice)

Definition 9. $OPT(j)$ = value of the optimal solution using only jobs $1, 2, \dots, j$.

For the last job j , there are exactly two cases:

1. **OPT does not select job j .** Then $OPT(j) = OPT(j - 1)$.
2. **OPT selects job j .** Then we collect profit v_j and can no longer use jobs $\{p(j) + 1, \dots, j - 1\}$. We must solve optimally for jobs $1, \dots, p(j)$. So $OPT(j) = v_j + OPT(p(j))$.

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\{v_j + OPT(p(j)), OPT(j - 1)\} & \text{otherwise} \end{cases}$$

This recurrence has the *optimal substructure property* (proved via exchange argument). This is a **binary choice**: either take job j or leave it.

3.2.4 Brute Force

Input: $n, s[1..n], f[1..n], v[1..n]$

Sort jobs by finish time so that $f[1] \leq f[2] \leq \dots \leq f[n]$.

Compute $p[1], p[2], \dots, p[n]$.

Compute-Opt(j)

if $j = 0$

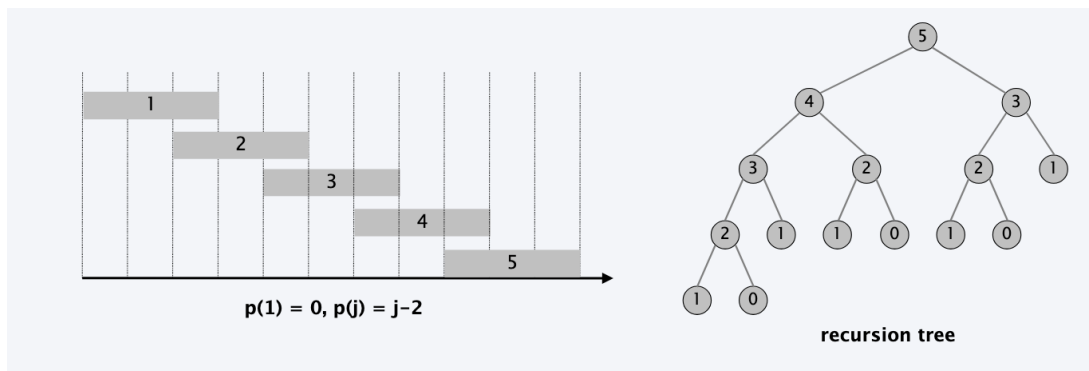
return 0.

else

return $\max(v[j] + \text{Compute-Opt}(p[j]), \text{Compute-Opt}(j-1))$.

A naive recursive implementation recomputes the same subproblems many times. For a

In pratica, $p(j) = j - 2$ significa che il lavoro j è compatibile solo con il lavoro $j - 2$ (mentre $j - 1$ si sovrappone a j e non può essere scelto insieme a j). Questo rende la ricorsione simile a quella dei numeri di Fibonacci, perché ogni soluzione dipende da due sottoproblemi distinti. $n = 5$, $p(5) = 3$ significa che il lavoro 5 è compatibile solo con il lavoro 3 (e non con il lavoro 4). Quindi, quando consideriamo se includere il lavoro 5, dobbiamo guardare alla soluzione ottimale per i primi 3 lavori ($\text{OPT}(3)$) invece che per i primi 4 lavori ($\text{OPT}(4)$). In questo caso, $\text{OPT}(5)$ dipende da $\text{OPT}(3)$ e $\text{OPT}(4)$, e $\text{OPT}(4)$ dipende da $\text{OPT}(2)$ e $\text{OPT}(3)$, e così via, creando una struttura ricorsiva che porta a molte chiamate ripetute degli stessi sottoproblemi.



3.2.5 Memoization: Top-Down DP

Key idea. Cache the result of each subproblem the first time it is computed, and look it up directly on future calls.

```

Input:  n, s[1..n], f[1..n], v[1..n]
Sort jobs by finish time so that  $f[1] \leq f[2] \leq \dots \leq f[n]$ .
Compute p[1], p[2], ..., p[n].

for j = 1 to n
    M[j]  $\leftarrow$  empty.
M[0]  $\leftarrow$  0.

M-Compute-Opt(j)
if M[j] is empty
    M[j]  $\leftarrow$  max( $v[j] + \text{M-Compute-Opt}(p[j])$ ,  $\text{M-Compute-Opt}(j - 1)$ ).
return M[j].

```

Claim 2. *The memoized algorithm runs in $O(n \log n)$ time.*

Proof. Sorting by finish time: $O(n \log n)$. Computing $p(\cdot)$: $O(n \log n)$ via binary search after sorting by start time. Each call to $\text{M-COMPUTE-OPT}(j)$ either (i) returns an existing value in $O(1)$, or (ii) fills in a new entry and makes two recursive calls. Let Φ = number of non-empty entries in M : initially $\Phi = 0$, and case (ii) increases Φ by 1. Since $\Phi \leq n$ throughout, there are at most $2n$ recursive calls of type (ii), so $\text{M-COMPUTE-OPT}(n)$ uses $O(n)$ time after preprocessing. $\square$

3.2.6 Finding the Actual Solution

The DP computes the optimal *value*. To recover the actual *set of jobs*, do a second pass:

```

Find-Solution(j)
if j = 0
    return  $\emptyset$ .
else if ( $v[j] + M[p[j]] > M[j-1]$ )
    return {j}  $\cup$  Find-Solution(p[j]).
else
    return Find-Solution(j-1).

```

At most n recursive calls, so $O(n)$ time.

3.2.7 Bottom-Up DP

Instead of recursion with memoization (top-down), we can fill the table iteratively from small to large (bottom-up):

BOTTOM-UP ($n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$)

Sort jobs by finish time so that $f_1 \leq f_2 \leq \dots \leq f_n$.

Compute $p(1), p(2), \dots, p(n)$.

$M[0] \leftarrow 0$.

FOR $j = 1$ **TO** n

$M[j] \leftarrow \max \{ v_j + M[p(j)], M[j-1] \}$.

Bottom-up and top-down (with memoization) produce the same table. The choice between them is largely a matter of personal style and the specific problem.

3.3 Segmented Least Squares

3.3.1 Background: Least Squares

Given n points $(x_1, y_1), \dots, (x_n, y_n)$ in the plane, the **least squares** line $y = ax + b$ minimises the sum of squared errors:

$$\text{SSE} = \sum_{i=1}^n (y_i - ax_i - b)^2.$$

Calculus gives the closed-form solution:

$$a = \frac{n \sum_i x_i y_i - (\sum_i x_i)(\sum_i y_i)}{n \sum_i x_i^2 - (\sum_i x_i)^2}, \quad b = \frac{\sum_i y_i - a \sum_i x_i}{n}.$$

3.3.2 Problem Definition

In practice, data often follows *several* line segments, not a single line.

Input. n points $(x_1, y_1), \dots, (x_n, y_n)$ with $x_1 < x_2 < \dots < x_n$, and a penalty constant $c > 0$.

Goal. Find a sequence of line segments that minimises:

$$f = E + cL,$$

where E is the total sum of squared errors across all segments, and L is the number of segments. The constant c controls the trade-off between *accuracy* (small E) and *parsimony* (small L).

3.3.3 Optimal Substructure (Multiway Choice)

Definition 10. • $e(i, j)$ = minimum sum of squared errors for points $p_i, p_{i+1}, \dots, p_j$ (the cost of fitting a single line to this segment).

- $OPT(j)$ = minimum cost for points $p_1, p_2, \dots, p_j$.

The last segment must end at p_j and start at some p_i . For each choice of i :

$$\text{cost} = e(i, j) + c + OPT(i - 1).$$

This gives the recurrence:

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \min_{1 \leq i \leq j} \{e(i, j) + c + OPT(i - 1)\} & \text{otherwise} \end{cases}$$

This is a **multiway choice**: we choose *which point* to start the last segment from, not just a binary yes/no.

Example

Consider four points sorted by their x -coordinates:

$$p_1 = (1, 1), \quad p_2 = (2, 2), \quad p_3 = (3, 2), \quad p_4 = (4, 4).$$

Let the penalty for each segment be $c = 1$.

Assume the following precomputed least-squares errors:

$$e_{1,1} = 0, \quad e_{1,2} = 0.5, \quad e_{1,3} = 1.2, \quad e_{1,4} = 3.0,$$

$$e_{2,2} = 0, \quad e_{2,3} = 0.2, \quad e_{2,4} = 1.5,$$

$$e_{3,3} = 0, \quad e_{3,4} = 0.4, \quad e_{4,4} = 0.$$

We define $OPT(j)$ as the minimum cost of approximating the first j points. The dynamic programming recurrence is

$$OPT(j) = \min_{1 \leq i \leq j} (e_{i,j} + c + OPT(i - 1)), \quad OPT(0) = 0.$$

Now compute the values:

$$OPT(1) = e_{1,1} + c + OPT(0) = 0 + 1 + 0 = 1,$$

$$OPT(2) = \min\{e_{1,2} + 1 + OPT(0), e_{2,2} + 1 + OPT(1)\} = \min\{1.5, 2\} = 1.5,$$

$$OPT(3) = \min\{e_{1,3} + 1 + OPT(0), e_{2,3} + 1 + OPT(1), e_{3,3} + 1 + OPT(2)\} = \min\{2.2, 2.2, 2.5\} = 2.2,$$

$$\begin{aligned} OPT(4) &= \min\{e_{1,4} + 1 + OPT(0), e_{2,4} + 1 + OPT(1), e_{3,4} + 1 + OPT(2), e_{4,4} + 1 + OPT(3)\} \\ &= \min\{4.0, 3.5, 3.9, 3.2\} = 3.2. \end{aligned}$$

Therefore, the optimal cost for all four points is $OPT(4) = 3.2$.

3.3.4 Algorithm

SEGMENTED-LEAST-SQUARES ($n, p_1, \dots, p_n, c$)

FOR $j = 1$ TO n

FOR $i = 1$ TO j

Compute the least squares $e(i, j)$ for the segment $p_i, p_{i+1}, \dots, p_j$.

$M[0] \leftarrow 0$.

FOR $j = 1$ TO n

$M[j] \leftarrow \min_{1 \leq i \leq j} \{ e_{ij} + c + M[i-1] \}$.

RETURN $M[n]$.

Theorem 4 (Bellman 1961). *The DP algorithm solves the segmented least squares problem in $O(n^3)$ time and $O(n^2)$ space.*

Proof. There are $O(n^2)$ pairs (i, j) . Computing $e(i, j)$ for each pair takes $O(n)$ time using the closed-form formula. The bottleneck is therefore $O(n^2) \times O(n) = O(n^3)$. $\square$

By precomputing prefix sums $\sum x_i, \sum y_i, \sum x_i y_i, \sum x_i^2$, each $e(i, j)$ can be computed in $O(1)$, reducing the total to $O(n^2)$ time and $O(n)$ space.

3.4 Knapsack Problem

3.4.1 Problem Definition

Input. n items, a knapsack of capacity W . Item i has weight $w_i > 0$ and value $v_i > 0$.

Goal. Choose a subset of items with total weight $\leq W$ that maximises total value.

Example (weight limit $W = 11$):

Item i	Value v_i	Weight w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

Optimal: items $\{3, 4\}$ with value $18 + 22 = 40$. **Greedy strategies** (by value, by weight, by ratio v_i/w_i) all fail to find this.

3.4.2 Why One Variable Is Not Enough

the problem is NP-complete, so we do not expect a polynomial-time algorithm in the input size. However, we can solve it in pseudo-polynomial time using dynamic programming. A first attempt defines $\text{OPT}(i) = \text{max profit from items } 1, \dots, i$. This does not work because when we add item i , we do not know the remaining capacity. We need to track *both* which items we have considered *and* how much capacity we have left.

3.4.3 Optimal Substructure (New Variable)

Definition 11. $\text{OPT}(i, w) = \text{maximum value from items } 1, \dots, i \text{ with weight limit } w$.

For item i :

1. **OPT does not take item i :** use best of items $1, \dots, i - 1$ with limit w .
2. **OPT takes item i :** gain v_i , reduce capacity to $w - w_i$, and use best of items $1, \dots, i - 1$ with new limit.

$$\text{OPT}(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ \text{OPT}(i - 1, w) & \text{if } w_i > w \\ \max\{\text{OPT}(i - 1, w), v_i + \text{OPT}(i - 1, w - w_i)\} & \text{otherwise} \end{cases}$$

in the second case, we reject item i because it is too heavy. In the third case, we have a binary choice: either reject item i and keep the same capacity, or accept item i , gain its value, and reduce the capacity accordingly. This is the “**adding a new variable**” technique: when one variable is not enough to capture all necessary information, add another.

3.4.4 Bottom-Up Algorithm

```
KNAPSACK( $n, W, w_1, \dots, w_n, v_1, \dots, v_n$ )  
  
FOR  $w = 0$  TO  $W$   
     $M[0, w] \leftarrow 0$ .  
  
FOR  $i = 1$  TO  $n$   
    FOR  $w = 1$  TO  $W$   
        IF ( $w_i > w$ )  $M[i, w] \leftarrow M[i - 1, w]$ .  
        ELSE  $M[i, w] \leftarrow \max \{ M[i - 1, w], v_i + M[i - 1, w - w_i] \}$ .  
  
RETURN  $M[n, W]$ .
```

Tracing the solution. After computing the table, item i is in the optimal solution for (i, w) if and only if $M[i, w] > M[i - 1, w]$.

Knapsack problem: bottom-up demo

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max\{OPT(i-1, w), v_i + OPT(i-1, w-w_i)\} & \text{otherwise} \end{cases}$$

		weight limit w											
		0	1	2	3	4	5	6	7	8	9	10	11
subset of items $1, \dots, i$	$\{\}$	0	0	0	0	0	0	0	0	0	0	0	0
	$\{1\}$	0	1	1	1	1	1	1	1	1	1	1	1
	$\{1, 2\}$	0	1	6	7	7	7	7	7	7	7	7	7
	$\{1, 2, 3\}$	0	1	6	7	7	18	19	24	25	25	25	25
	$\{1, 2, 3, 4\}$	0	1	6	7	7	18	22	24	28	29	29	40
	$\{1, 2, 3, 4, 5\}$	0	1	6	7	7	18	22	28	29	34	35	40

$OPT(i, w) = \text{max profit subset of items } 1, \dots, i \text{ with weight limit } w.$

3.4.5 Running Time and Remarks

Theorem 5. The knapsack DP algorithm runs in $\Theta(nW)$ time and $\Theta(nW)$ space.

Proof. Each of the $\Theta(nW)$ table entries is filled in $O(1)$ time. □

Important remark. The running time $\Theta(nW)$ is *not* polynomial in the input size. The input size is $O(n \log W)$ (since W is encoded in binary), so the algorithm is **pseudo-polynomial**. In fact:

- The decision version of knapsack is **NP-complete** (Chapter 8).
- There exists a polynomial-time approximation scheme (PTAS) that finds a solution within 1% of the optimum (Section 11.8).

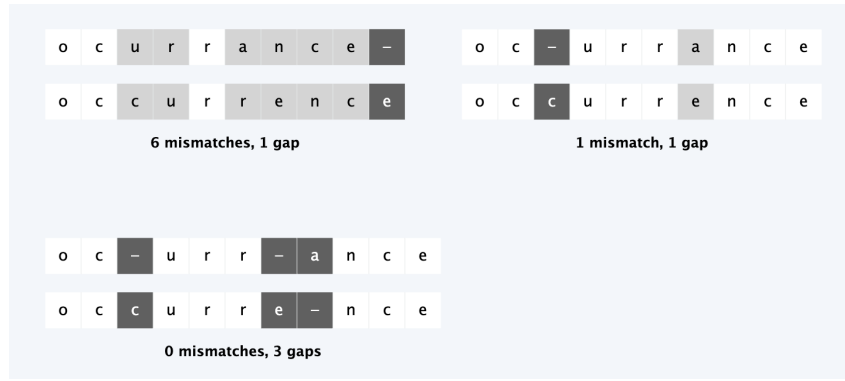
in particular: $if(v_i < V - w_i < W)$

$$I(n, w) = \sum_{i=1}^n \log w_i + \log v_i + \log W == O(n * \max(\log W, \log V)) = O(n \log W)$$

3.5 Sequence Alignment

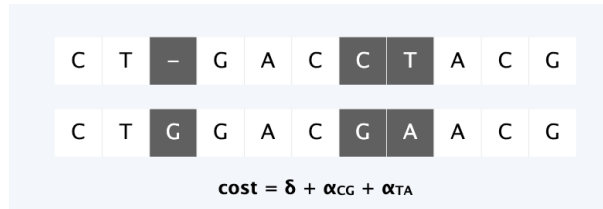
3.5.1 String Similarity and Edit Distance

A natural question in computer science and biology is: *how similar are two strings?* Consider the strings occurrence and occurrence. Depending on how we align them, we get very different costs:



The **edit distance** (Levenshtein 1966, Needleman–Wunsch 1970) measures this cost formally using two parameters:

- δ = gap penalty (cost of leaving a character unmatched)
- α_{pq} = mismatch penalty for aligning character p with character q (equals 0 if $p = q$)



Applications: Unix `diff`, spell checking, speech recognition, computational biology (DNA/protein alignment), and more.

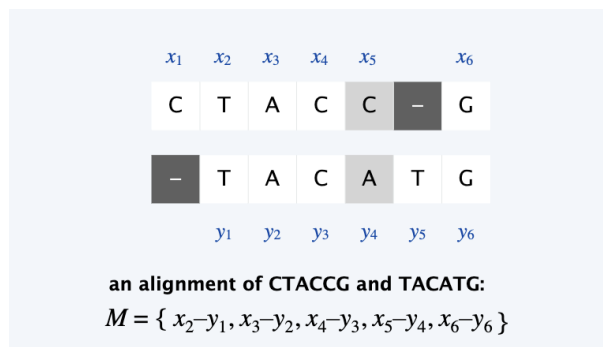
3.5.2 Problem Definition

Definition 12 (Alignment). Given strings $x_1x_2\ldots x_m$ and $y_1y_2\ldots y_n$, an alignment M is a set of ordered pairs (x_i, y_j) such that each character appears in at most one pair, and there are no crossings (i.e., if $(x_i, y_j) \in M$ and $(x_{i'}, y_{j'}) \in M$ with $i < i'$, then $j < j'$).

Definition 13 (Cost of an alignment).

$$\text{cost}(M) = \underbrace{\sum_{(x_i, y_j) \in M} \alpha_{x_i y_j}}_{\text{mismatch}} + \underbrace{\delta \sum_{i: x_i \text{ unmatched}} 1 + \delta \sum_{j: y_j \text{ unmatched}} 1}_{\text{gaps}}$$

Goal. Find the alignment of minimum cost.



3.5.3 Optimal Substructure

Definition 14. $OPT(i, j)$ = minimum cost of aligning the prefix strings $x_1 \dots x_i$ and $y_1 \dots y_j$.

There are three cases for what happens to the last characters x_i and y_j :

1. x_i **is matched with** y_j : pay mismatch $\alpha_{x_i y_j}$, then solve the subproblem on $x_1 \dots x_{i-1}$ and $y_1 \dots y_{j-1}$.
2. x_i **is left unmatched (gap)**: pay δ , then solve on $x_1 \dots x_{i-1}$ and $y_1 \dots y_j$.
3. y_j **is left unmatched (gap)**: pay δ , then solve on $x_1 \dots x_i$ and $y_1 \dots y_{j-1}$.

This gives the recurrence:

$$OPT(i, j) = \begin{cases} j\delta & \text{if } i = 0 \\ i\delta & \text{if } j = 0 \\ \min \{ \alpha_{x_i y_j} + OPT(i-1, j-1), \delta + OPT(i-1, j), \delta + OPT(i, j-1) \} & \text{otherwise} \end{cases}$$

in the first two cases, we have to pay a gap penalty for each unmatched character because one of the 2 strings is empty. In the third case, we have a **three-way choice**: we can match x_i with y_j , or leave x_i unmatched, or leave y_j unmatched.

3.5.4 Algorithm

SEQUENCE-ALIGNMENT ($m, n, x_1, \dots, x_m, y_1, \dots, y_n, \delta, \alpha$)

FOR $i = 0$ **TO** m

$M[i, 0] \leftarrow i\delta$.

FOR $j = 0$ **TO** n

$M[0, j] \leftarrow j\delta$.

FOR $i = 1$ **TO** m

FOR $j = 1$ **TO** n

$M[i, j] \leftarrow \min \{ \alpha[x_i, y_j] + M[i-1, j-1], \\ \delta + M[i-1, j], \\ \delta + M[i, j-1] \}.$

RETURN $M[m, n]$.

Theorem 6. *The DP algorithm computes the edit distance of two strings of length m and n in $\Theta(mn)$ time and $\Theta(mn)$ space.*

Recovering the alignment. To find not just the cost but also the actual alignment, trace back through the DP table from $M[m, n]$ to $M[0, 0]$, following the choice that was made at each step.

Space optimisation. We can compute the optimal *value* in $O(mn)$ time and only $O(m + n)$ space by keeping just two rows of the table at a time. However, this makes it hard to recover the actual alignment.

3.6 Hirschberg's Algorithm

3.6.1 Goal: Linear Space with Full Alignment

Theorem 7. *There exists an algorithm to find an optimal alignment in $O(mn)$ time and $O(m + n)$ space.*

This is achieved by a clever combination of divide-and-conquer and dynamic programming (Hirschberg, 1975). The key insight is to model the problem as a shortest-path problem.

Edit Distance Graph

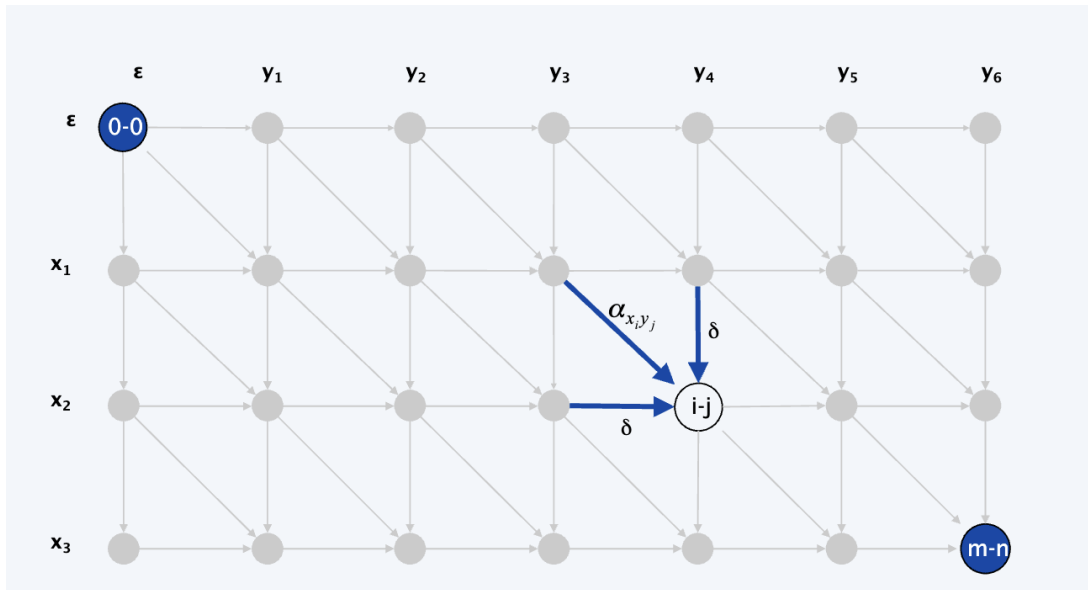


Figure 3.1: The edit distance problem can be represented as a shortest path problem in a directed acyclic graph. Each node corresponds to a pair of indices (i, j) , and edges represent the three possible operations (match/mismatch(diagonale), gap in x , gap in y (verticale/orizzontale)) with their associated costs. The optimal alignment corresponds to the shortest path from $(0, 0)$ to (m, n) .

Lemma 1. *Let $f(i, j)$ be the cost of the shortest path from $(0, 0)$ to (i, j) . Then $f(i, j) = OPT(i, j)$ for all i, j .*

Proof (by strong induction on $i + j$). Base case: $f(0, 0) = \text{OPT}(0, 0) = 0$. Inductive step: the last edge into (i, j) comes from $(i-1, j-1)$, $(i-1, j)$, or $(i, j-1)$. Applying the inductive hypothesis to each predecessor:

$$f(i, j) = \min\{\alpha_{x_i y_j} + f(i-1, j-1), \delta + f(i-1, j), \delta + f(i, j-1)\} = \text{OPT}(i, j). \quad \square$$

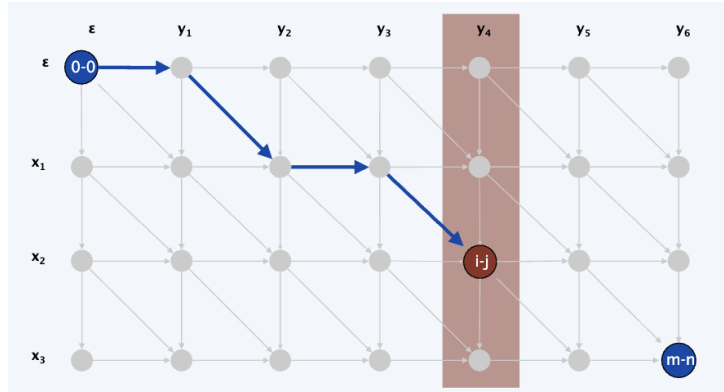
□

Forward and Backward Distances

Define two functions:

- $f(i, j)$ = shortest path from $(0, 0)$ to (i, j) (computed *forward*).
- $g(i, j)$ = shortest path from (i, j) to (m, n) (computed *backward*, by reversing all edges and swapping the roles of $(0, 0)$ and (m, n)).

Both $f(\cdot, j)$ and $g(\cdot, j)$ can be computed for any fixed column j in $O(mn)$ time and $O(m + n)$ space.



Key observations:

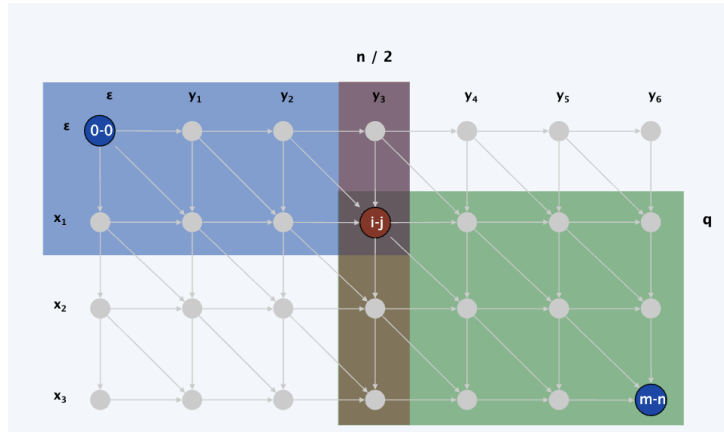
1. The total cost of the shortest path through node (i, j) is $f(i, j) + g(i, j)$.
2. Let q be the index that minimises $f(q, \lfloor n/2 \rfloor) + g(q, \lfloor n/2 \rfloor)$. Then the optimal alignment passes through node $(q, \lfloor n/2 \rfloor)$.

In other words, the optimal alignment is not just any path through the graph: it must pass through a specific node in the middle column. This key observation makes the divide-and-conquer approach possible, because it guarantees that some node in the middle column belongs to the optimal solution, and we can identify it by combining the forward and backward dynamic programming computations.

3.6.2 Divide-and-Conquer

1. **Divide.** Compute $f(\cdot, n/2)$ (forward) and $g(\cdot, n/2)$ (backward). Find index q minimising $f(q, n/2) + g(q, n/2)$. The optimal alignment passes through $(q, n/2)$.
2. **Conquer.** Recursively compute the optimal alignment in:
 - the top-left subproblem: $x_1 \dots x_q$ vs. $y_1 \dots y_{n/2}$

- the bottom-right subproblem: $x_{q+1} \dots x_m$ vs. $y_{n/2+1} \dots y_n$



Example of Hirschberg's Algorithm

Consider the strings $x = AC$ and $y = AGC$.

The goal is to find an optimal alignment between these two strings. Hirschberg's algorithm first splits the second string in the middle. Since y has length 3, the middle column is $\lfloor 3/2 \rfloor = 1$, so we look at the first character A.

Now we do two computations:

- a forward computation from the start to the middle column,
- a backward computation from the end to the middle column.

For each possible row q , we add the forward cost and the backward cost. The best value tells us which node in the middle column belongs to the optimal alignment.

In this example, the best path passes through the node that matches the first A. After that, the problem is split into two smaller parts:

- the left part, before the middle column,
- the right part, after the middle column.

This example shows the main idea of Hirschberg's algorithm: find one correct point in the middle, then solve the two halves recursively. The final result is the same optimal alignment as the standard dynamic programming algorithm, but with much less space.

3.6.3 Running Time Analysis

Theorem 8. *Hirschberg's algorithm runs in $O(mn)$ time and $O(m + n)$ space.*

Proof (by induction on n). Let $T(m, n)$ be the running time. The divide step takes $O(mn)$ time. The two subproblems are of sizes $(q, n/2)$ and $(m - q, n/2)$. Choose constant

c such that the divide step costs at most cmn . Claim: $T(m, n) \leq 2cmn$.

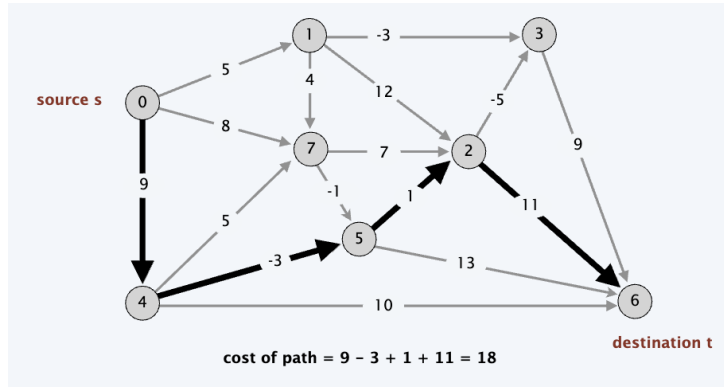
$$\begin{aligned}
 T(m, n) &\leq T(q, n/2) + T(m - q, n/2) + cmn \\
 &\leq 2cq(n/2) + 2c(m - q)(n/2) + cmn \\
 &= cqn + cmn - cqn + cmn \\
 &= 2cmn. \quad \square
 \end{aligned}$$

The key point is that the two subproblems split the *rows* (q and $m - q$), so the total work across all levels of recursion stays $O(mn)$. $\square$

3.7 Bellman-Ford Algorithm

3.7.1 The Shortest Path Problem with Negative Weights

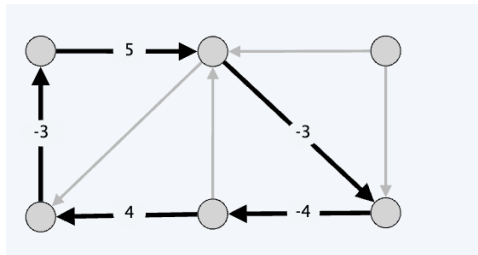
Problem. Given a directed graph $G = (V, E)$ with arbitrary edge weights c_{vw} , find the cheapest path from every node v to a destination node t .



Why Dijkstra fails. Dijkstra's algorithm requires non-negative edge weights. With negative weights it can give wrong answers. A simple fix (adding a constant to all edges) also fails because it changes the lengths of paths with different numbers of edges.

3.7.2 Negative Cycles

Definition 15 (Negative cycle). A negative cycle is a directed cycle W such that $\sum_{e \in W} c_e < 0$.



Lemma 2. If some path from v to t contains a negative cycle, then there is no cheapest path from v to t (the cost can be made arbitrarily negative).

Lemma 3. *If G has no negative cycles, then there exists a cheapest $v \rightsquigarrow t$ path that is simple (uses at most $n - 1$ edges).*

Proof. Take a cheapest path with the fewest edges. If it contains a cycle W , removing W gives a cheaper or equal-cost path with fewer edges (since $c(W) \geq 0$). Contradiction. $\square$

Shortest path and negative cycle problems

- Shortest path problem. Given a digraph $G = (V, E)$ with edge weights c_{vw} and no negative cycles, find cheapest v to t path for each node v .
- Negative cycle problem. Given a digraph $G = (V, E)$ with edge weights c_{vw} , determine whether G contains a negative cycle.

The two problems are closely related: if we can solve the shortest path problem, we can check for negative cycles by seeing if any path cost can be reduced by going around a cycle. Conversely, if we can solve the negative cycle problem, we can find shortest paths by first checking for negative cycles and then applying a shortest path algorithm that assumes no negative cycles.

3.7.3 Dynamic Programming Formulation

Definition 16. $OPT(i, v) = \text{cost of the cheapest } v \rightsquigarrow t \text{ path using at most } i \text{ edges.}$

Recurrence: (è come se partissi da t e facessi passi indietro, ogni passo indietro può essere o restare fermi o prendere un arco in ingresso)

$$OPT(i, v) = \begin{cases} 0 & \text{if } i = 0 \text{ and } v = t \\ \infty & \text{if } i = 0 \text{ and } v \neq t \\ \min\{OPT(i-1, v), \min_{(v,w) \in E} \{OPT(i-1, w) + c_{vw}\}\} & \text{otherwise} \end{cases}$$

The two cases are: (1) the cheapest path uses $\leq i - 1$ edges (do not move yet), or (2) it uses exactly i edges (take the first edge (v, w) and then find the best $w \rightsquigarrow t$ path with $\leq i - 1$ edges).

Key observation. If G has no negative cycles, then $OPT(n-1, v)$ gives the cost of the cheapest $v \rightsquigarrow t$ path (by Lemma 2, the cheapest path is simple and has at most $n - 1$ edges).

3.7.4 Implementation

```

SHORTEST-PATHS( $V, E, c, t$ )
  FOR each node  $v$  in  $V$ :
     $M[0, v] = \text{infinity}$ 
   $M[0, t] = 0$ 
  FOR  $i = 1$  TO  $n-1$ :
    FOR each node  $v$  in  $V$ :
       $M[i, v] = M[i-1, v]$ 
      FOR each edge  $(v, w)$  in  $E$ :
         $M[i, v] = \min(M[i, v], M[i-1, w] + c(v, w))$ 
  RETURN  $M$ 

```

Theorem 9. *Given a digraph with no negative cycles, the DP algorithm computes the cost of the cheapest $v \rightsquigarrow t$ path for each node v in $\Theta(mn)$ time and $\Theta(n^2)$ space.*

3.7.5 Bellman-Ford: Efficient Implementation

In practice, we use only $O(n)$ space by maintaining a single array $d(v)$ (the best distance found so far) and a successor pointer.

```

BELLMAN-FORD( $V, E, c, t$ )
  FOR each node  $v$  in  $V$ :
     $d(v) = \text{infinity}$ 
     $\text{successor}(v) = \text{null}$ 
   $d(t) = 0$ 
  FOR  $i = 1$  TO  $n-1$ :
    FOR each node  $w$  in  $V$ :
      IF  $d(w)$  was updated in previous iteration:
        FOR each edge  $(v, w)$  in  $E$ :
          IF  $d(v) > d(w) + c(v, w)$ :
             $d(v) = d(w) + c(v, w)$ 
             $\text{successor}(v) = w$ 
    IF no  $d(w)$  value changed in this iteration: STOP

```

Performance optimisation. If $d(w)$ was not updated in iteration $i - 1$, there is no reason to consider edges entering w in iteration i . This can make the algorithm much faster in practice.

3.7.6 Analysis and Correctness

Lemma 4. *Throughout Bellman-Ford, $d(v)$ is the cost of some $v \rightsquigarrow t$ path. After pass i , $d(v) \leq \text{OPT}(i, v)$.*

Proof (by induction on i). After pass $i + 1$, for any $v \rightsquigarrow t$ path P with $i + 1$ edges, let (v, w) be the first edge and P' be the subpath from w to t . By the inductive hypothesis, $d(w) \leq c(P')$. After considering v in pass $i + 1$:

$$d(v) \leq c_{vw} + d(w) \leq c_{vw} + c(P') = c(P). \quad \square$$

□

Theorem 10. *Given a digraph with no negative cycles, Bellman-Ford computes the costs of the cheapest $v \rightsquigarrow t$ paths in $O(mn)$ time and $\Theta(n)$ extra space.*

Caution. The claim that “following successor pointers always gives a path of cost $d(v)$ ” is *false* in general during the algorithm (the successor graph may contain cycles or have inconsistent costs). However, upon termination with no negative cycles, the successor graph is a tree and following pointers from any s gives the correct shortest path.

Lemma 5. *If the successor graph contains a directed cycle W at any point during Bellman-Ford, then W is a negative cycle.*

Proof. When $\text{successor}(v) = w$ is set, we have $d(v) \geq d(w) + c_{vw}$. Let $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$ be the cycle. Summing the inequalities (with a strict inequality for the last edge

added to the cycle) gives:

$$c(v_1, v_2) + c(v_2, v_3) + \cdots + c(v_k, v_1) < 0. \quad \square$$

□

Theorem 11. *Given a digraph with no negative cycles, Bellman-Ford finds the cheapest $s \rightsquigarrow t$ paths in $O(mn)$ time and $\Theta(n)$ extra space.*

Proof. By Lemma 4, the successor graph has no cycle on termination. Following successor pointers from s gives a path $s = v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_k = t$. Upon termination, for each $\text{successor}(v) = w$, we have $d(v) = d(w) + c_{vw}$ (since values no longer change). Summing along the path: $d(s) = d(t) + \sum_{\text{edges}} c = c(P)$. Combined with Theorem 2, $c(P)$ is the minimum cost. □

3.8 Distance Vector Protocols

3.8.1 Routing in Communication Networks

In a communication network, nodes are routers, edges are direct links, and edge weights represent link delays. We want each router to know the shortest path to every other router.

Dijkstra needs global knowledge of the whole network. **Bellman-Ford** needs only local knowledge of neighbouring nodes — each router only talks to its neighbours.

3.8.2 Distance Vector Protocols

Each router maintains:

- A vector of shortest known distances to every other node.
- The first hop (direction) on each shortest path.

The algorithm runs n separate Bellman-Ford computations, one for each destination. Updates propagate through the network by “*routing by rumour*”: each router shares its distance vector with its neighbours, and they update their own vectors accordingly.

Examples: RIP, Cisco’s IGRP, AppleTalk’s RTMP, Novell’s IPX RIP.

Asynchrony. The algorithm still converges even if routers do not update in lockstep.

3.8.3 Counting-to-Infinity Problem

Problem. If a link is deleted, distance vector protocols can get stuck in a loop where routers keep increasing their estimated distances, very slowly converging.

Example. If the link $s \rightarrow t$ (cost 1) is deleted, s thinks it can reach t via v (cost 2), then v updates via s (cost 3), and so on — slowly counting to infinity.

3.8.4 Path Vector Protocols

Link-state routing fixes the counting-to-infinity problem by storing the full path (not just the distance and first hop). This allows routers to detect loops immediately. Requires much more storage. Used in **BGP** (internet routing) and **OSPF**.

3.9 Negative Cycle Detection

3.9.1 The Problem

Goal. Given a directed graph $G = (V, E)$ with edge weights c_{vw} , find a negative cycle if one exists.

Application: Currency Arbitrage. Given n currencies and exchange rates between pairs, is there a sequence of conversions that makes a profit?

Example. $\text{USD} \rightarrow \text{EUR} \rightarrow \text{GBP} \rightarrow \text{USD}$ with rates $0.741 \times 1.366 \times 0.995 = 1.00714 > 1$: this is an arbitrage opportunity. Modelling this as a graph problem (with $-\log(\text{exchange rate})$ as edge weights), arbitrage corresponds to a negative cycle.

3.9.2 Detection via Bellman-Ford

Lemma 6. *If $\text{OPT}(n, v) = \text{OPT}(n-1, v)$ for all v , then there is no negative cycle.*

Lemma 7. *If $\text{OPT}(n, v) < \text{OPT}(n-1, v)$ for some node v , then the cheapest $v \rightsquigarrow t$ path has exactly n edges. By the pigeonhole principle, it contains a directed cycle W . Since removing W would give a shorter path, W must be a negative cycle.*

3.9.3 Algorithm

Theorem 12. *A negative cycle can be found in $\Theta(mn)$ time and $\Theta(n^2)$ space.*

Proof. Add a new node t and connect every node to t with a 0-cost edge. The new graph G' has a negative cycle if and only if G has one. Run the DP for n iterations (instead of $n-1$). If $\text{OPT}(n, v) = \text{OPT}(n-1, v)$ for all v : no negative cycle. Otherwise, extract the directed cycle from the path from v to t (the cycle cannot contain t since no edges leave t). $\square$

Theorem 13. *A negative cycle can be found in $O(mn)$ time and $O(n)$ extra space.*

Proof. Run Bellman-Ford for n passes (not $n-1$) on the modified digraph G' .

- If no $d(v)$ is updated in pass n : no negative cycle.
- Otherwise, suppose $d(s)$ is updated in pass n . Define $\text{pass}(v)$ = last pass in which $d(v)$ was updated. We have $\text{pass}(s) = n$ and $\text{pass}(\text{successor}(v)) \geq \text{pass}(v) - 1$. Following successor pointers from s , we must eventually revisit a node. By Lemma 4, the resulting cycle is a negative cycle. $\square$

$\square$

3.9.4 Summary of Complexities

Algorithm	Problem	Time	Space
Sequence Alignment (basic DP)	Edit distance + alignment	$\Theta(mn)$	$\Theta(mn)$
Sequence Alignment (space opt.)	Edit distance only	$O(mn)$	$O(m + n)$
Hirschberg's algorithm	Edit distance + alignment	$O(mn)$	$O(m + n)$
Shortest Paths (basic DP)	No negative cycles	$\Theta(mn)$	$\Theta(n^2)$
Bellman-Ford (efficient)	No negative cycles	$O(mn)$	$\Theta(n)$
Negative Cycle Detection	Find neg. cycle	$\Theta(mn)$ / $O(mn)$	$\Theta(n^2)$ / $O(n)$

Chapter 4

exercises on dynamic programming

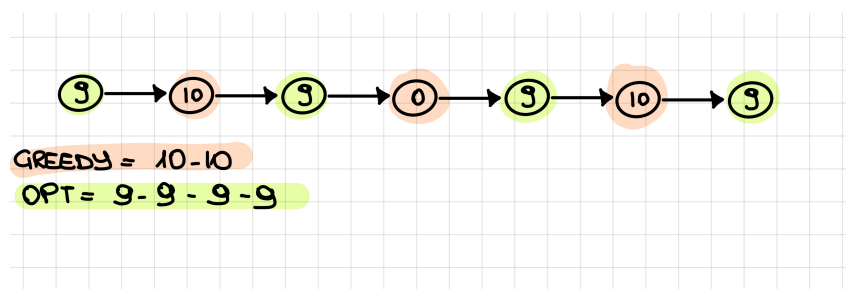
Problem 1 (Max Independent Set on Line Graphs)

Let $G = (V, E)$ be a line graph of n nodes formed by a sequence of nodes $v_1, v_2, \dots, v_n$ with an edge between v_i and v_j if and only if i and j differs by exactly 1. With every vertex i , we associate a positive integer weight w_i . Recall that a set of nodes is an *independent set* if no two nodes of them are joined by an edge. The goal is to find an independent set of G of maximum total weight.

- (a) Show on an example that a greedy strategy fails to find an optimal solution.
- (b) Give an algorithm that finds an independent set of maximum total weight in time which is polynomial in the number of vertices and it does not depend from the weights of the nodes.
- (c) Provide the pseudo-code of an efficient implementation of the algorithm and discuss its time and space complexity.

solution

- (a) the greedy strategy fails to find an optimal solution. For example, consider the line graph with 7 nodes and weights $w_1 = 9, w_2 = 10, w_3 = 9, w_4 = 0, w_5 = 9, w_6 = 10, w_7 = 9$. The greedy strategy would pick the greater value so it would pick v_2 and v_6 with total weight 20. However, the optimal solution is to pick v_1, v_3, v_5 , and v_7 with total weight 36.



- (b) We can solve this problem using dynamic programming. We define $\text{OPT}(i)$ as the maximum total weight of an independent set that can be formed from the first i

nodes.

$$\text{OPT}(i) = \begin{cases} 0 & \text{se } i = 0 \\ w_1 & \text{se } i = 1 \\ \max\{\text{OPT}(i-1), \text{OPT}(i-2) + w_i\} & \text{se } i \geq 2 \end{cases}$$

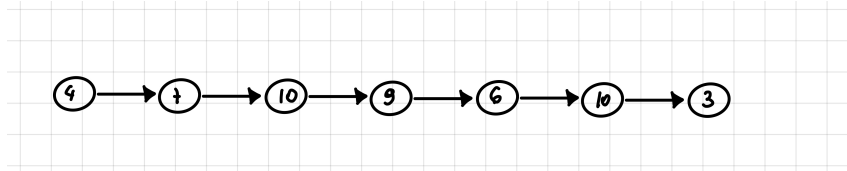
(c) The pseudo-code of the algorithm is as follows:

```

def max_independent_set(weights):
    n = len(weights)
    OPT[0] = 0
    OPT[1] = weights[1]
    for i =2 to n:
        if weights[i] + OPT[i - 2] > OPT[i - 1]:
            OPT[i] = weights[i] + OPT[i - 2]
        else:
            OPT[i] = OPT[i - 1]
    return OPT[n]

```

example of the algorithm in action:



$$\begin{aligned}
 \text{OPT}(0) &= 0 \\
 \text{OPT}(1) &= 4 \\
 \text{OPT}(2) &= \max\{\text{OPT}(1), \text{OPT}(0) + 7\} = \max\{4, 7\} = 7 \\
 \text{OPT}(3) &= \max\{\text{OPT}(2), \text{OPT}(1) + 10\} = \max\{7, 14\} = 14 \\
 \text{OPT}(4) &= \max\{\text{OPT}(3), \text{OPT}(2) + 9\} = \max\{14, 16\} = 16 \\
 \text{OPT}(5) &= \max\{\text{OPT}(4), \text{OPT}(3) + 6\} = \max\{16, 20\} = 20 \\
 \text{OPT}(6) &= \max\{\text{OPT}(5), \text{OPT}(4) + 10\} = \max\{20, 26\} = 26 \\
 \text{OPT}(7) &= \max\{\text{OPT}(6), \text{OPT}(5) + 3\} = \max\{26, 23\} = 26
 \end{aligned}$$

algorithm to know which nodes are in the optimal solution:

```

def find_solution(OPT, weights, i):
    i = len(weights)
    if i=0:
        return []
    if i=1:
        return [1]
    if weights[i] + OPT[i - 2] > OPT[i - 1]:
        return find_solution(OPT, weights, i - 2) + [i]

```

```

else :
    return find_solution(OPT, weights , i - 1)

```

Problem 2 (Rod Cutting)

We are given a rod of integer length L . We assume that a rod of integer length $0 \leq l \leq L$ has a known value $v(l)$. The problem is to find the best way of cutting the rod, such that the sum of the values of the resulting rods is maximised.

- (a) Show a dynamic programming algorithm that solves the problem.
- (b) Prove the correctness of the recursive form used, and the optimality of the solution returned by the algorithm.
- (c) Provide the pseudo-code of an efficient implementation of the algorithm and discuss its time complexity.

Solution

- (a) We can solve this problem using dynamic programming. Let $v(i)$ be the value of a rod of length i , and $R(l)$ be the maximum value obtainable for a rod of length l .

The optimal substructure of the problem allows us to define the following recurrence:

$$R(l) = \begin{cases} 0 & \text{if } l = 0 \\ \max_{1 \leq j \leq l} \{v(j) + R(l - j)\} & \text{if } l > 0 \end{cases}$$

- (b) **Proof of Correctness:** An optimal cut for a rod of length l consists of a first piece of length j and an optimal arrangement of the remaining $l - j$ length. Since the value of the first piece $v(j)$ is fixed, to maximize the total value we must maximize the value of the remaining part, which is $R(l - j)$. By iterating over all possible first cuts $j \in \{1, \dots, l\}$, we are guaranteed to find the global maximum.

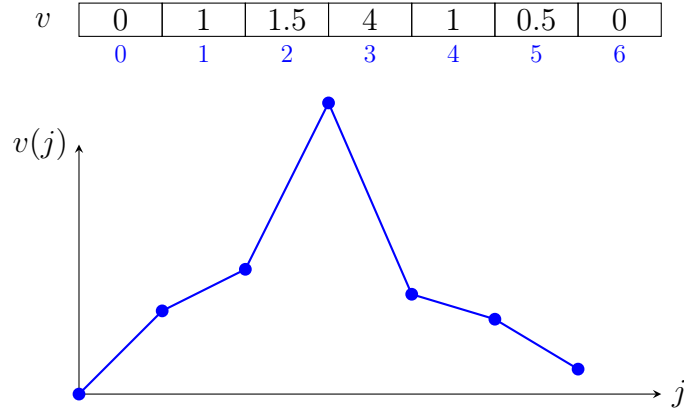


Table $R[l]$ (Max Values)

0	1	2	4	5	6	8
0	1	2	3	4	5	6

$$R(0) = 0$$

$$R(1) = 1$$

$$R(2) = 2 \quad \max\{v(1) + R(1), v(2) + R(0)\}$$

Scelta ottima: $j = 1$

$$R(3) = 4 \quad \max\{v(1) + R(2), v(2) + R(1), v(3) + R(0)\}$$

$\nearrow$
 $v(3) = 4$
 Scelta ottima: $j = 3$

$\nwarrow$
 $R(0) = 0$

(c) **Implementation and Complexity:**

First, we consider the basic *bottom-up* implementation that computes only the maximum revenue $R[L]$:

Listing 4.1: Basic Bottom-Up Rod Cutting

```
def rod_cutting(v, L):
    R = [0] * (L + 1)
    for j in range(1, L + 1):
        q = float('-inf')
        for k in range(1, j + 1):
            q = max(q, v[k] + R[j - k])
        R[j] = q
    return R[L]
```

To reconstruct the actual cuts used to achieve this maximum, we extend the algorithm by using an auxiliary array S to store the length of the first piece i that achieved the maximum revenue for each subproblem j :

Listing 4.2: Extended Bottom-Up Rod Cutting

```
def extended_bottom_up_rod_cutting(v, L):
    R = [0] * (L + 1)
    S = [0] * (L + 1)
    for j in range(1, L + 1):
        q = float('-inf')
        for i in range(1, j + 1):
```

```

        if q < v[i] + R[j - i]:
            q = v[i] + R[j - i]
            S[j] = i
    R[j] = q
return R, S

```

Complexity Analysis:

- **Time Complexity:** $O(L^2)$. The nested loops result in a summation $\sum_{j=1}^L j = \frac{L(L+1)}{2}$, which is quadratic.
- **Space Complexity:** $O(L)$ to store the memoization table R and the optimal cuts S .

Solution Reconstruction: To print the lengths of the pieces in an optimal solution for length L :

```

def print_optimal_cuts(S, L):
    while L > 0:
        print(S[L])
        L = L - S[L]

```

Chapter 5

Network Flow

5.1 Max-Flow and Min-Cut Problems

Max-flow and min-cut are two fundamental problems in network flow theory, defined on a directed graph $G = (V, E)$ with a source node s , a sink node t , and a capacity function $c : E \rightarrow \mathbb{R}^+$ that assigns a non-negative capacity to each edge.

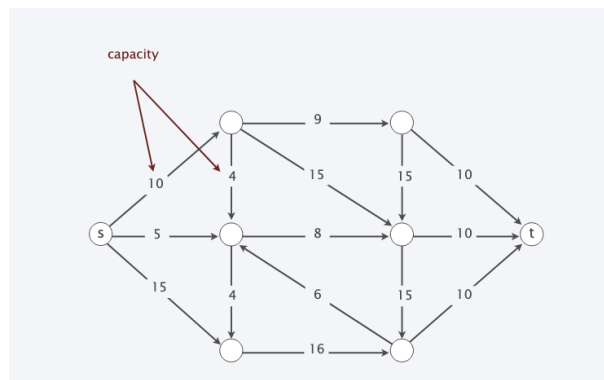


Figure 5.1: Example of a flow network with source s , sink t , and edge capacities.

Definition 17. An st -cut (*cut*) is a partition (A, B) of the vertices such that $s \in A$ and $t \in B$.

Definition 18. The capacity of an st -cut (A, B) is the sum of the capacities of edges going from A to B :

$$\text{cap}(A, B) = \sum_{\substack{e = (u, v) \\ u \in A, v \in B}} c(e).$$

Edges from B to A are **not** counted.

Min-cut problem. Find an st -cut of minimum capacity.

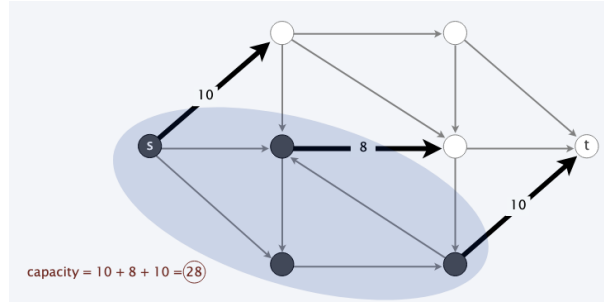


Figure 5.2: capacity of 28.

Definition 19. An st-flow is a function $f : E \rightarrow \mathbb{R}^+$ satisfying:

- **Capacity constraint:** For every edge $e \in E$, $0 \leq f(e) \leq c(e)$. un arco non può trasportare più del suo limite di capacità.
- **Flow conservation:** For every vertex $v \in V \setminus \{s, t\}$, the total flow into v equals the total flow out of v :

$$\sum_{\substack{e=(u,v) \\ (u,v) \in E}} f(e) = \sum_{\substack{e=(v,w) \\ (v,w) \in E}} f(e).$$

un nodo che non è né la sorgente né il pozzo non può accumulare flusso, quindi tutto quello che entra deve uscire.

Definition 20. The value of a flow f is

$$\text{val}(f) = \sum_{e \text{ out of } s} f(e),$$

i.e., the total flow leaving the source s .

Max-flow problem. Find an st-flow of maximum value.

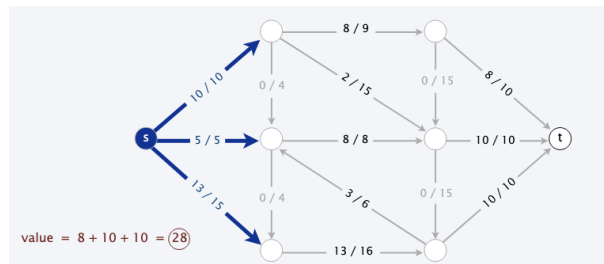


Figure 5.3: Example of a maximum flow with value 28.

5.2 Ford-Fulkerson Algorithm

5.2.1 Greedy Approach and Its Limits

A natural greedy strategy is:

- Start with $f(e) = 0$ for all edges $e \in E$.
- Find an st-path P where every edge e satisfies $f(e) < c(e)$ (i.e., has remaining capacity).

- Push as much flow as possible along P (limited by the minimum remaining capacity on the path).
- Repeat until no such path exists.

Problem with the greedy approach. The order in which augmenting paths are chosen can cause the algorithm to terminate with a sub-optimal flow. For instance, on certain networks the greedy approach reaches a flow of value 16, even though the true maximum is 19. This happens because saturating some edges blocks all remaining paths from s to t , even when additional flow could theoretically be routed differently.

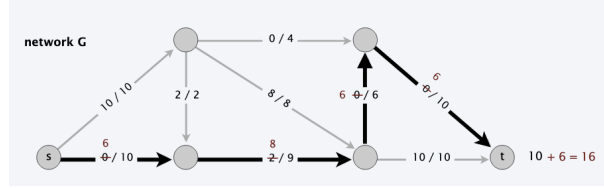


Figure 5.4: Along the augmenting path $s \rightarrow 2 \rightarrow 4 \rightarrow t$, the bottleneck is 8 (limited by edge $2 \rightarrow 4$, capacity 8), bringing the total flow to 8. Edge $s \rightarrow 2$ still has $10 - 8 = 2$ units of residual capacity, which are routed along path $s \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow t$ through edge $2 \rightarrow 3$ (capacity 2), saturating $s \rightarrow 2$ and raising the total flow to 10. Finally, the augmenting path $s \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow t$ has bottleneck 6, yielding a total flow of $10 + 6 = 16$.

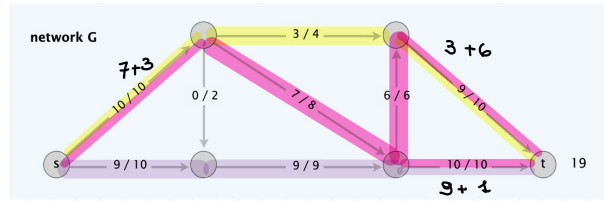


Figure 5.5:

The solution is to use a *residual graph* that allows the algorithm to *undo* previously sent flow, which is the core idea behind Ford-Fulkerson.

5.2.2 Residual Graph

Definition 21. Given a flow network $G = (V, E)$ and a flow f , the residual graph $G_f = (V, E_f)$ is constructed as follows. For each original edge $e = (u, v) \in E$ with capacity $c(e)$ and flow $f(e)$, we create:

- A forward residual edge $e = (u, v)$ with residual capacity $c_f(e) = c(e) - f(e)$, representing the remaining capacity (included only if $c_f(e) > 0$, i.e., $f(e) < c(e)$).



Figure 5.6: The forward residual edge from u to v has capacity $c(e) - f(e)$.

- A backward residual edge $e^R = (v, u)$ with residual capacity $c_f(e^R) = f(e)$, representing the flow that can be cancelled (included only if $f(e) > 0$).

Formally:

$$E_f = \{e \in E : f(e) < c(e)\} \cup \{e^R : e \in E, f(e) > 0\},$$

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \in E, \\ f(e) & \text{if } e^R \in E. \end{cases}$$

Key property. A flow f' in G_f corresponds to a valid flow $f + f'$ in the original graph G .

5.2.3 Augmenting Path

Definition 22. An augmenting path is a simple st -path P in the residual graph G_f such that every edge on P has strictly positive residual capacity.

Definition 23. The bottleneck capacity of an augmenting path P in G_f is

$$\text{bottleneck}(G_f, P) = \min_{e \in P} c_f(e),$$

i.e., the minimum residual capacity among all edges on P . It is the maximum amount of additional flow that can be pushed along P without violating any capacity constraint.

Key property. Let f be a flow in G and P be an augmenting path in G_f . Let $b = \text{bottleneck}(G_f, P)$. After augmenting along P :

$$\text{val}(f') = \text{val}(f) + b.$$

AUGMENT (f, c, P)

$b \leftarrow$ bottleneck capacity of path P .

FOREACH edge $e \in P$

IF ($e \in E$) $f(e) \leftarrow f(e) + b$.

ELSE $f(e^R) \leftarrow f(e^R) - b$.

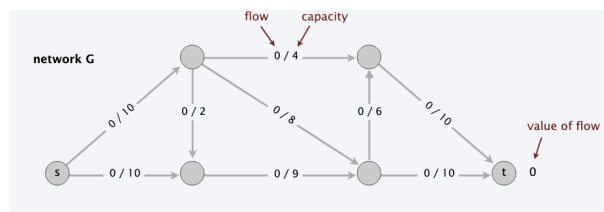
RETURN f .

5.2.4 Ford-Fulkerson Algorithm

```

FORD-FULKERSON( $G, s, t, c$ )
  FOREACH edge  $e \in E : f(e) \leftarrow 0$ .
   $G_f \leftarrow$  residual graph.
  WHILE (there exists an augmenting path  $P$  in  $G_f$ )
     $f \leftarrow$  AUGMENT( $f, c, P$ ).
    Update  $G_f$ .
  RETURN  $f$ .
  }
```

example

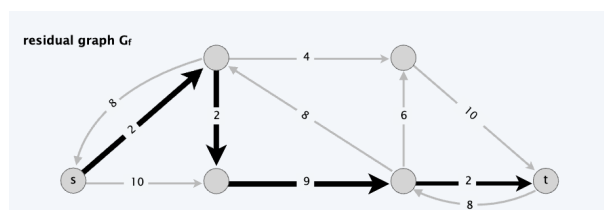


Step 1. Find an augmenting path P_1 in the residual graph G_f . The bottleneck capacity is 8. Augment along P_1 :

$$\text{val}(f) = 0 + 8 = 8.$$

Step 2. Find a new augmenting path P_2 in the updated residual graph. The bottleneck capacity is 2. Augment along P_2 :

$$\text{val}(f) = 8 + 2 = 10.$$



Step 3. Find augmenting path P_3 with bottleneck capacity 6:

$$\text{val}(f) = 10 + 6 = 16.$$

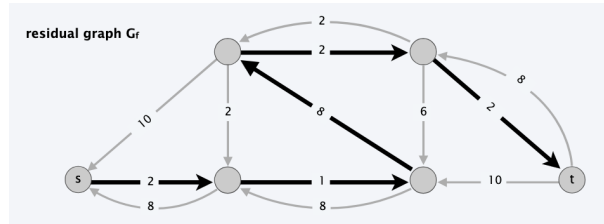
At this point, a purely greedy algorithm (without the residual graph) would get stuck: no forward-edge-only path from s to t remains. However, the residual graph G_f still contains augmenting paths that use **backward edges** to “reroute” flow already sent.

Step 4 — use of a backward edge. In G_f , there exists a path P_4 that traverses a backward (reverse) residual edge. Sending flow along a backward edge *cancels* part of a previously assigned flow, effectively rerouting it. The bottleneck capacity is 2:

$$\text{val}(f) = 16 + 2 = 18.$$

Step 5. One further augmenting path P_5 with bottleneck capacity 1:

$$\text{val}(f) = 18 + 1 = 19.$$



After step 5 there is no augmenting path in G_f , so the algorithm terminates.

5.3 Max-Flow Min-Cut Theorem

The Max-Flow Min-Cut Theorem establishes a fundamental duality between flows and cuts: the maximum value of any st-flow equals the minimum capacity of any st-cut. We build towards this result in three steps.

5.3.1 Flow Value Lemma

Lemma 8 (Flow Value Lemma). *Let f be any st-flow and let (A, B) be any st-cut. Then the net flow across (A, B) equals the value of f :*

$$\text{val}(f) = \sum_{\substack{e=(u,v) \\ u \in A, v \in B}} f(e) - \sum_{\substack{e=(u,v) \\ u \in B, v \in A}} f(e).$$

Proof. By definition, $\text{val}(f) = \sum_{e \text{ out of } s} f(e)$. We extend the sum over all nodes in A :

$$\text{val}(f) = \sum_{v \in A} \left(\sum_{e \text{ out of } v} f(e) - \sum_{e \text{ into } v} f(e) \right).$$

By **flow conservation**, every term with $v \neq s$ vanishes. Rearranging the remaining terms by whether the edge crosses the cut ($A \rightarrow B$) or goes backwards ($B \rightarrow A$) yields the result. $\square$

Edges entirely within A or within B cancel out in the sum, so only edges *crossing* the cut contribute to $\text{val}(f)$.

5.3.2 Weak Duality

Corollary 1 (Weak Duality). *For any st-flow f and any st-cut (A, B) :*

$$\text{val}(f) \leq \text{cap}(A, B).$$

Proof. Using the Flow Value Lemma and the capacity constraint $f(e) \leq c(e)$:

$$\text{val}(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ into } A} f(e) \leq \sum_{e \text{ out of } A} f(e) \leq \sum_{e \text{ out of } A} c(e) = \text{cap}(A, B).$$

□

Weak duality immediately implies that any flow value is an *upper bound* on every cut capacity, and vice versa. In particular, if we ever find a flow f and a cut (A, B) such that $\text{val}(f) = \text{cap}(A, B)$, both must be simultaneously optimal.

5.4 Max-Flow Min-Cut Theorem

Theorem 14 (Max-Flow Min-Cut Theorem). *In any flow network, the value of a maximum st-flow equals the capacity of a minimum st-cut:*

$$\max_f \text{val}(f) = \min_{(A,B)} \text{cap}(A, B).$$

Proof. We prove the equivalence of the following three conditions for any flow f :

- (i) There exists an st-cut (A, B) such that $\text{cap}(A, B) = \text{val}(f)$.
- (ii) f is a maximum flow.
- (iii) There is no augmenting path in the residual graph G_f .

(i) $\Rightarrow$ (ii). Suppose (A, B) is a cut with $\text{cap}(A, B) = \text{val}(f)$. For any other flow f' , weak duality gives $\text{val}(f') \leq \text{cap}(A, B) = \text{val}(f)$, so f is a maximum flow.

(ii) $\Rightarrow$ (iii). We prove the contrapositive. If there exists an augmenting path P in G_f , then we can strictly increase $\text{val}(f)$ by pushing $\text{bottleneck}(G_f, P) > 0$ additional units of flow along P . Hence f is not a maximum flow.

(iii) $\Rightarrow$ (i). Suppose no augmenting path exists in G_f . Define:

$$A = \{v \in V : v \text{ is reachable from } s \text{ in } G_f\}.$$

By definition, $s \in A$. Since there is no augmenting path, $t \notin A$, so $(A, B = V \setminus A)$ is a valid st-cut.

Now consider any edge $e = (u, v)$ with $u \in A, v \in B$:

- The forward residual edge (u, v) cannot be in G_f (otherwise v would be reachable), so $f(e) = c(e)$.
- Any edge $e' = (v, u)$ with $v \in B, u \in A$: the backward residual edge (u, v) cannot be in G_f , so $f(e') = 0$.

Therefore, applying the Flow Value Lemma:

$$\text{val}(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ into } A} f(e) = \sum_{e \text{ out of } A} c(e) - 0 = \text{cap}(A, B).$$

□

5.4.1 Augmenting Path Theorem

A direct consequence of the proof above is the following algorithmic certificate:

Corollary 2 (Augmenting Path Theorem). *A flow f is a maximum flow if and only if there is no augmenting path in the residual graph G_f .*

This corollary is the theoretical foundation of the Ford-Fulkerson algorithm: the algorithm terminates precisely when f is optimal, and at that point the set of nodes reachable from s in G_f defines the corresponding minimum cut.

Running Time

Assumption All edge capacities are integers between 1 and C . All edge capacities are integers between 1 and C .

Lemma 9 (Integrality Invariant). *Throughout the Ford-Fulkerson algorithm, all flow values $f(e)$ and all residual capacities $c_f(e)$ remain integers.*

Proof. Initially all flows are 0. Each augmentation increases the flow along a path by $\text{bottleneck}(G_f, P)$, which is the minimum of a finite set of integer residual capacities, hence an integer. By induction, all values remain integers throughout. □

Theorem 15. *The Ford-Fulkerson algorithm terminates in at most $\text{val}(f^*) \leq nC$ iterations.*

Proof. By the integrality invariant, every augmentation increases $\text{val}(f)$ by at least 1. Since $\text{val}(f^*)$ is the maximum possible flow value and $\text{val}(f^*) \leq nC$ (at most n edges out of s , each with capacity at most C), the algorithm terminates after at most nC iterations. □

Corollary 3. *The running time of Ford-Fulkerson is $O(m \cdot n \cdot C)$, since each iteration requires $O(m)$ time to find an augmenting path via DFS or BFS.*

Corollary 4. *If $C = 1$, the running time of Ford-Fulkerson reduces to $O(mn)$.*

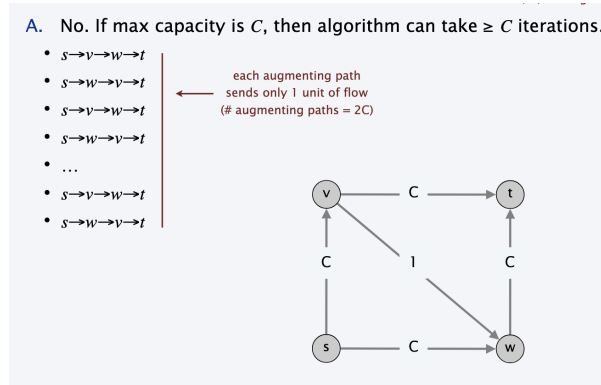
Theorem 16 (Integrality Theorem). *If all edge capacities are integers, then there exists a maximum flow f^* in which every flow value $f^*(e)$ is an integer.*

Proof. Since all capacities are integers, the integrality invariant guarantees that the flow f maintained by the algorithm is always integer-valued. Because the algorithm terminates (by the previous theorem), the flow upon termination is both a maximum flow and integer-valued. □

Ford-Fulkerson is **not** a polynomial-time algorithm in general: its complexity $O(mnC)$ depends on the *value* of the capacities, not just on the *size* of the input. In the worst case, if C is exponentially large, the algorithm takes exponentially many iterations. For polynomial guarantees, variants such as **Edmonds-Karp** (shortest augmenting path, $O(m^2n)$) or **capacity scaling** ($O(m^2 \log C)$) must be used.

5.5 Capacity-Scaling Algorithm

The basic Ford-Fulkerson algorithm can be very slow if it picks augmenting paths with small bottleneck capacity. The **capacity-scaling algorithm** fixes this by always working with large-capacity paths first.



Key Idea

Instead of using the full residual graph, we work on a restricted subgraph $G_f(\Delta)$ that contains only edges with residual capacity $\geq \Delta$. We start with Δ equal to the largest power of 2 that is $\leq C$ (the maximum capacity), and we halve Δ at each phase.

CAPACITY-SCALING(G, s, t, c)

FOREACH edge $e \in E : f(e) \leftarrow 0$.

$\Delta \leftarrow$ largest power of 2 $\leq C$.

WHILE ($\Delta \geq 1$)

$G_f(\Delta) \leftarrow \Delta$ -residual graph.

WHILE (there exists an augmenting path P in $G_f(\Delta)$)

$f \leftarrow$ **AUGMENT** (f, c, P).

 Update $G_f(\Delta)$.

$\Delta \leftarrow \Delta / 2$.

RETURN f .

Correctness

When $\Delta = 1$, the Δ -residual graph equals the full residual graph. So when the algorithm terminates, there are no augmenting paths in G_f , which means f is a max-flow by the

Max-Flow Min-Cut theorem.

Running Time Analysis

- **Lemma 1.** The outer while loop runs at most $1 + \lfloor \log_2 C \rfloor$ times, since Δ starts at most C and halves each time.
- **Lemma 2.** At the end of a Δ -scaling phase, the value of the max-flow satisfies $\text{val}(f^*) \leq \text{val}(f) + 2m\Delta$.
- **Lemma 3.** There are at most $2m$ augmentations per scaling phase. Each augmentation in a Δ -phase increases the flow by at least Δ , and by Lemma 2, at most $2m\Delta$ units remain to be added.

Theorem 17. *The capacity-scaling algorithm finds a max-flow using $O(m \log C)$ augmentations and runs in $O(m^2 \log C)$ time.*

Proof of Lemma 2

At the end of a Δ -scaling phase, let A be the set of nodes reachable from s in $G_f(\Delta)$. Then (A, B) is a cut, and every edge from A to B has residual capacity $< \Delta$, so:

$$\text{val}(f^*) \leq \text{val}(f) + \sum_{e \text{ out of } A} \Delta \leq \text{val}(f) + m\Delta \leq \text{val}(f) + 2m\Delta.$$

5.6 Shortest Augmenting Paths

Another way to improve Ford-Fulkerson is to always pick the augmenting path with the **fewest edges**, found using BFS.

SHORTEST-AUGMENTING-PATH(G, s, t, c)

FOREACH $e \in E : f(e) \leftarrow 0$.

$G_f \leftarrow$ residual graph.

WHILE (there exists an augmenting path in G_f)

$P \leftarrow$ **BREADTH-FIRST-SEARCH** (G_f, s, t).

$f \leftarrow$ **AUGMENT** (f, c, P).

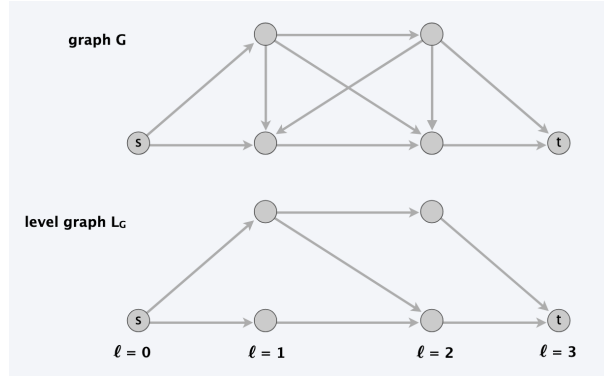
Update G_f .

RETURN f .

5.6.1 Level Graph

The **level graph** L_G of a graph G assigns to each node v a level $\ell(v)$ equal to the number of edges in the shortest path from s to v . It contains only edges (v, w) where $\ell(w) = \ell(v) + 1$.

A path is a shortest s - v path in G if and only if it is a path in L_G . The level graph can be computed in $O(m + n)$ time by running BFS and deleting all backward and side edges.



Analysis

- **L1. The shortest path length never decreases.** Let f and f' be the flows before and after an augmentation, and let L, L' be the corresponding level graphs. When we augment along a shortest path, the only new edges added to $G_{f'}$ are *backward* edges (reversals of edges used). A backward edge (w, v) goes from a higher level to a lower level, so any path using it is strictly longer than the current shortest path. Therefore the shortest path length in $G_{f'}$ is at least that in G_f .
- **L2. After at most m augmentations, the shortest path strictly increases.** Each augmentation saturates at least one edge — the bottleneck — which is then removed from L_G . The only new edges added to L_G are backward edges, which go in the wrong direction and cannot be part of a shortest path of the same length. Therefore, after at most m augmentations, all edges at the current length are exhausted and the shortest path must strictly increase.

Theorem 18. *The shortest augmenting path algorithm runs in $O(m^2n)$ time.*

Proof. • Each BFS to find a shortest augmenting path takes $O(m + n)$ time.

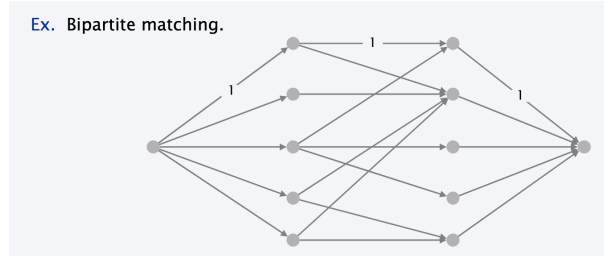
- By L2, there are at most m augmentations for each fixed path length.
- By L1, the path length can increase at most $n - 1$ times (it starts at 1 and is at most $n - 1$).
- Total augmentations: $O(m \cdot n)$.
- Total time: $O(mn) \cdot O(m + n) = O(m^2n)$.

□

Remark. The bound $O(m^2n)$ can be improved. By explicitly maintaining the level graph and processing all augmentations at the same length together (**blocking flow**), one phase takes $O(mn)$ time instead of $O(m^2)$, giving **Dinic's algorithm** with total running time $O(mn^2)$. Using dynamic trees (Sleator–Tarjan, 1983), this further improves to $O(mn \log n)$.

5.7 Unit-Capacity Simple Networks

A network is **unit-capacity simple** if every edge has capacity 1, and every non-source, non-sink node has either at most one entering edge or at most one leaving edge. Bipartite matching graphs are an important example.



Faster Analysis for This Special Case

Theorem 19 (Even–Tarjan, 1975). *In unit-capacity simple networks, the shortest augmenting path algorithm computes a max-flow in $O(m\sqrt{n})$ time.*

The proof uses three lemmas:

- **L1.** Each phase of normal augmentations takes $O(m)$ time.
- **L2.** After at most $\sqrt{n}$ phases, $f^* - f \leq \sqrt{n}$. After $\sqrt{n}$ phases the shortest path has length $> \sqrt{n}$, so the level graph has more than $\sqrt{n}$ layers. By a pigeonhole argument, some layer V_h has at most $\sqrt{n}$ nodes, giving a cut of capacity $\leq \sqrt{n}$.
- **L3.** After at most $\sqrt{n}$ additional augmentations (one unit each), the flow is optimal.

Combining: $O(\sqrt{n})$ phases $\times O(m)$ per phase + $O(\sqrt{n} \cdot m)$ final augmentations = $O(m\sqrt{n})$.

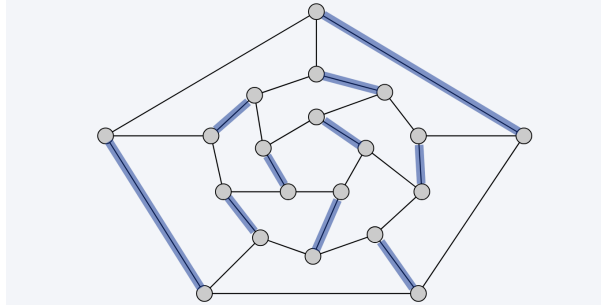
Application to Bipartite Matching

Since bipartite matching graphs are unit-capacity simple networks, the shortest augmenting path algorithm solves bipartite matching in $O(m\sqrt{n})$ time. This matches the bound of the **Hopcroft–Karp algorithm** (1973).

5.8 Bipartite Matching

A **matching** in a graph $G = (V, E)$ is a subset of edges $M \subseteq E$ such that each node appears in at most one edge. A graph is **bipartite** if the nodes can be split into two sets L and R so that every edge goes from L to R . The goal of bipartite matching is to find a matching of maximum cardinality.

Max matching. Given a graph, find a max cardinality matching.

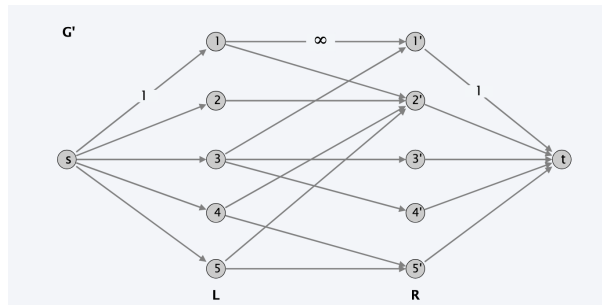


5.8.1 Reduction to Max Flow

Build a directed graph G' as follows:

- Direct all edges from L to R with capacity 1.
- Add a source s with unit capacity edges to every node in L .
- Add a sink t with unit capacity edges from every node in R .

Theorem 20. *The maximum cardinality of a matching in G equals the value of the maximum flow in G' .*



Proof. **(Flow $\Rightarrow$ Matching).** Let f be a maximum flow in G' . Since all capacities are 1, the integrality theorem guarantees an integer-valued max flow. The edges with $f(e) = 1$ form a matching in G of size $\text{val}(f)$, since each node can have at most one unit of flow passing through it. **(Matching $\Rightarrow$ Flow).** Let M be a maximum matching in G . We can construct a flow f in G' by sending 1 unit of flow along the path $s \rightarrow u \rightarrow v \rightarrow t$ for each edge $(u, v) \in M$. This flow has value equal to the size of the matching, $\text{val}(f) = |M|$. $\square$

5.8.2 Hall's Theorem

A **perfect matching** is a matching where every node appears in exactly one edge. For a bipartite graph with $|L| = |R|$, a perfect matching exists if and only if for every subset $S \subseteq L$, the set of neighbors $N(S)$ satisfies $|N(S)| \geq |S|$. This is **Hall's theorem**.

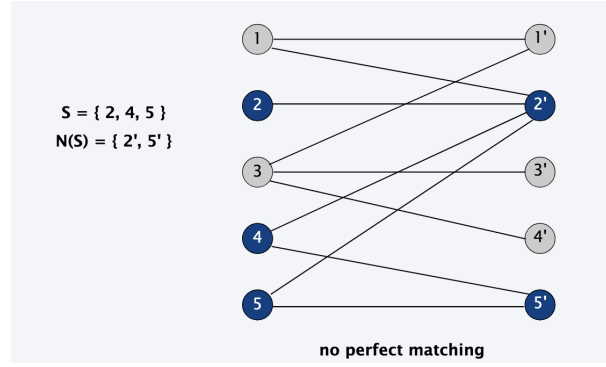


Figure 5.7: in this case, we choose as $S = \{2, 4, 5\}$ and $N(S) = \{2', 5'\}$; so $|N(S)| = 2 < |S| = 3$, for this reason, is impossible to find a perfect matching

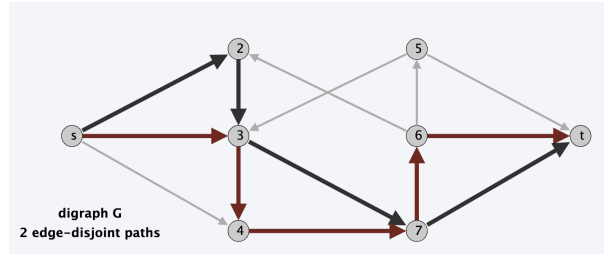
Theorem 21 (Hall, 1935). *A bipartite graph $G = (L \cup R, E)$ with $|L| = |R|$ has a perfect matching if and only if $|N(S)| \geq |S|$ for all $S \subseteq L$.*

As a corollary, every k -regular bipartite graph has a perfect matching, since the flow $f(u, v) = 1/k$ on each edge is a feasible flow of value $|L|$.

5.8.3 Edge-Disjoint Paths

Two paths are **edge-disjoint** if they share no edge in common.

The problem is: given a digraph G and two nodes s and t , find the maximum number of edge-disjoint s - t paths.



5.8.4 Reduction to Max Flow

Assign unit capacity to every edge. Then:

Theorem 22. *The maximum number of edge-disjoint s - t paths equals the value of the maximum flow.*

Proof. (Flow $\Rightarrow$ Paths). Let f be a maximum flow. By the integrality theorem, there exists a maximum flow f^* where every flow value is an integer (0 or 1). The edges with $f^*(e) = 1$ form a collection of edge-disjoint s - t paths, since each node can have at most one unit of flow passing through it. **(Paths $\Rightarrow$ Flow).** Let $P_1, \dots, P_k$ be a maximum collection of edge-disjoint s - t paths. We can construct a flow f by sending 1 unit of flow along each path P_i . This flow has value k , which is the number of edge-disjoint paths. $\square$

A direct consequence is **Menger's theorem**: the maximum number of edge-disjoint s - t paths equals the minimum number of edges whose removal disconnects t from s .

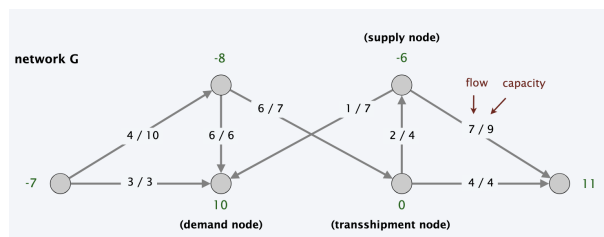
For **undirected** graphs, replace each undirected edge with two antiparallel directed edges of capacity 1. There always exists a maximum flow where at most one of each antiparallel pair carries flow, so the same result holds.

5.8.5 Circulation with Demands

A **circulation** is a generalization of flow where each node v has a demand d_v : a positive d_v means v needs to receive d_v more units than it sends, and a negative d_v means v produces flow.

A feasible circulation satisfies:

- **Capacity:** $0 \leq f(e) \leq c_e$ for each edge e .
- **Conservation:** $\sum_{e \text{ into } v} f(e) - \sum_{e \text{ out of } v} f(e) = d_v$ for each node v .

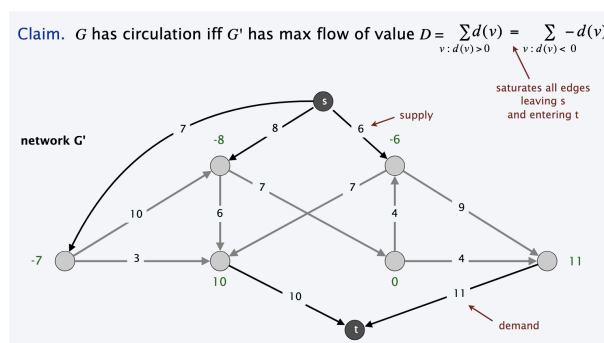


Reduction to Max Flow

Add a super-source s^* and super-sink t^* :

- For each node v with $d_v > 0$, add edge (s^*, v) with capacity d_v .
- For each node v with $d_v < 0$, add edge (v, t^*) with capacity $-d_v$.

A feasible circulation exists if and only if the max flow in this new graph saturates all edges leaving s^* .



Theorem 23. *Integrality theorem: if all capacities and demands are integers, then there exists a feasible circulation where every flow value is an integer.*

Theorem 24 (Cut certificate of infeasibility). *Given a circulation instance (V, E, c, d) , no feasible circulation exists if there is a partition (A, B) of V such that*

$$\sum_{v \in B} d_v > \text{cap}(A, B),$$

where

$$\text{cap}(A, B) = \sum_{\substack{(u,v) \in E \\ u \in A, v \in B}} c_{uv}.$$

5.8.6 Lower Bounds

Edges can also have a **lower bound** ℓ_e : the flow must satisfy $\ell_e \leq f(e) \leq c_e$. This can be reduced to the standard circulation problem by sending ℓ_e units along each edge in advance and updating the demands of both endpoints accordingly.

A **circulation** with lower bounds is a flow that satisfies :

- **Capacity:** $\ell_e \leq f(e) \leq c_e$ for each edge e .
- **Conservation:** $\sum_{e \text{ into } v} f(e) - \sum_{e \text{ out of } v} f(e) = d_v$ for each node v .

Simple idea. First, satisfy all lower bounds in advance: send ℓ_e units on every edge e . Then only the *extra* flow is unknown.

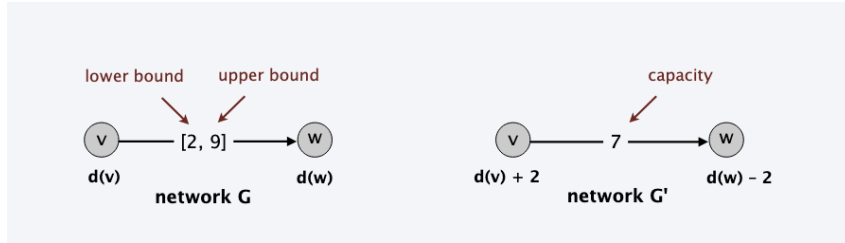
Define:

$$f'(e) = f(e) - \ell_e.$$

So in the transformed network G' :

- each edge has residual capacity interval $0 \leq f'(e) \leq c_e - \ell_e$;
- node demands must be updated because of the pre-sent flow:

$$d'_v = d_v + \sum_{e \text{ out of } v} \ell_e - \sum_{e \text{ into } v} \ell_e.$$



Theorem 25 (Reduction for lower bounds). *There exists a feasible circulation in G (with bounds $\ell_e \leq f(e) \leq c_e$) if and only if there exists a feasible circulation in the transformed instance G' (with bounds $0 \leq f'(e) \leq c_e - \ell_e$ and demands d'_v). Moreover, if all data are integers, then a feasible circulation can be chosen integer-valued.*

Proof sketch. Set $f'(e) = f(e) - \ell_e$ (equivalently, $f(e) = f'(e) + \ell_e$).

The capacity constraints are exactly the same information written in two different ways:

$$\ell_e \leq f(e) \leq c_e \iff 0 \leq f'(e) \leq c_e - \ell_e.$$

At each node v , replace f with $f' + \ell$ in flow conservation. This gives the new demand equation with d'_v . So feasibility in G and feasibility in G' are equivalent.

If capacities, demands, and lower bounds are integers, then G' has an integer feasible circulation (integrality theorem). Adding back ℓ_e keeps everything integer, so G also has an integer feasible circulation. $\square$

5.9 Survey Design

A company wants to survey n_1 consumers about n_2 products. Consumer i can only be asked about product j if they own it. Each consumer i must be asked between c_i^- and c_i^+ questions, and each product j must receive between p_j^- and p_j^+ responses.

This is modeled as a **circulation problem with lower bounds**:

- Add an edge (i, j) if consumer i owns product j .
- Add edges from s to each consumer and from each product to t , with lower and upper bounds given by the constraints.
- Add a return edge from t to s .

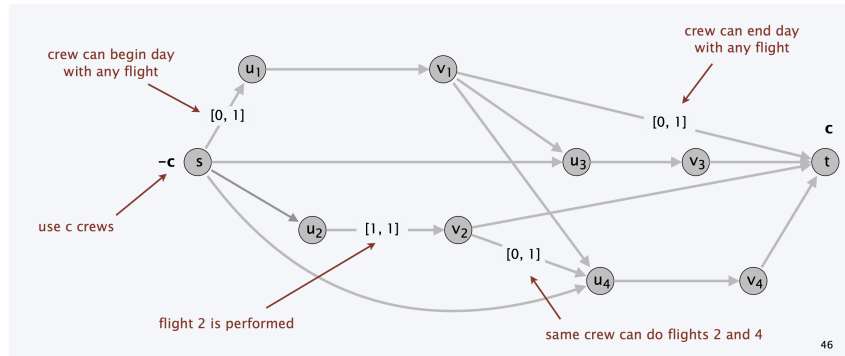
A feasible integer circulation corresponds to a valid survey design.

5.10 Airline Scheduling

The goal is to manage a set of k flights using the minimum number of crews. A crew can move from flight i to flight j if j is reachable after i ends (same or nearby airport, enough time).

This is modeled as a **circulation problem**:

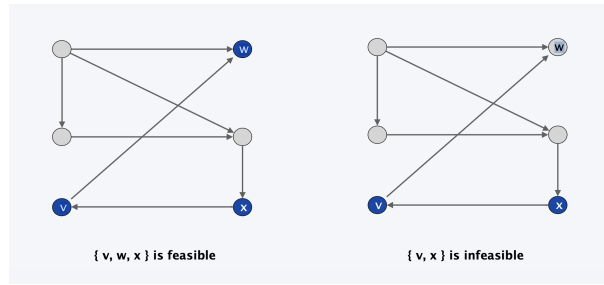
- For each flight i , add nodes u_i (departure) and v_i (arrival) with edge (u_i, v_i) of lower bound and capacity 1.
- If a crew can go from flight i to flight j , add edge (v_i, u_j) with capacity 1.
- Add source s and sink t to control the total number of crews.



Theorem 26. *The airline scheduling problem can be solved in $O(k^3 \log k)$ time, using binary search on the number of crews and solving $O(\log k)$ circulation problems.*

5.10.1 Project Selection

Each project v has a revenue p_v (which can be negative, representing a cost). Some projects require others: if project v requires project w , then v cannot be chosen without also choosing w . The goal is to find a feasible subset of projects that maximizes total revenue. **prerequisite edges** (v, w) indicate that v requires w .

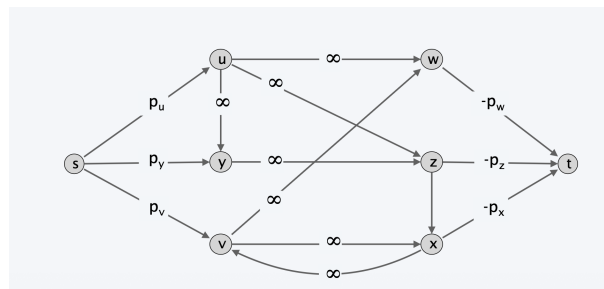


Reduction to Min Cut

Build a graph G' :

- For each prerequisite edge (v, w) , add an edge with ∞ capacity.
- For each project v with $p_v > 0$, add edge (s, v) with capacity p_v .
- For each project v with $p_v < 0$, add edge (v, t) with capacity $-p_v$.

The infinite capacity edges ensure that the selected set is always feasible. The minimum cut in G' gives the maximum revenue subset.



Chapter 6

exercises on Network Flow

Problem 1 (Escape Problem)

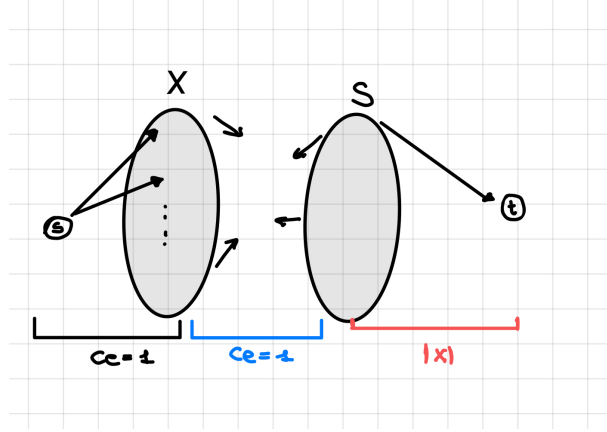
We define the Escape Problem as follows. We are given a directed graph $G = (V, E)$ (picture a network of roads). A certain collection of nodes $X \subset V$ are designated as populated nodes, and a certain other collection $S \subset V$ are designated as safe nodes. (Assume that X and S are disjoint.) In case of an emergency, we want evacuation routes from the populated nodes to the safe nodes. A set of evacuation routes is defined as a set of paths in G so that (i) each node in X is the tail of one path, (ii) the last node on each path lies in S , and (iii) the paths do not share any edges. Such a set of paths gives a way for the occupants of the populated nodes to “escape” to S , without overly congesting any edge in G .

- (a) Given G , X , and S , show how to decide in polynomial time whether such a set of evacuation routes exists.
- (b) Suppose we have the same problem as in (a), but we want to enforce an even stronger version of the “no congestion” condition (iii). Thus, we change (iii) to say “the paths do not share any nodes (except maybe the final one in S).” With this new condition, show how to decide in polynomial time whether such a set of evacuation routes exists.
- (c) Provide an example with the same G , X , and S , in which the answer is yes to the question in (a) but no to the question in (b).

Part (i): Edge-Disjoint Evacuation Routes

Construction of G' . We build a flow network $G' = (V', E')$ as follows:

- **Source edges:** For each $u \in X$, add edge (s, u) with capacity $c(s, u) = 1$.
- **Sink edges:** For each $v \in S$, add edge (v, t) with capacity $c(v, t) = |X|$. Multiple evacuation routes are allowed to end at the same safe node, so we do not restrict the flow entering t from S .
- **Internal edges:** For every original edge $e \in E$, keep it with capacity $c(e) = 1$.



A valid set of evacuation routes exists if and only if $\text{maxflow} = |X|$.

Proof. We prove both directions.

($\Rightarrow$) Given $|X|$ edge-disjoint paths, set $f(e) = 1$ if edge e is used by some path and $f(e) = 0$ otherwise. Edge-disjointness ensures all capacity constraints hold. Since one path leaves each $u \in X$, the edge (s, u) is saturated for all $u \in X$, so $\text{val}(f) = |X|$.

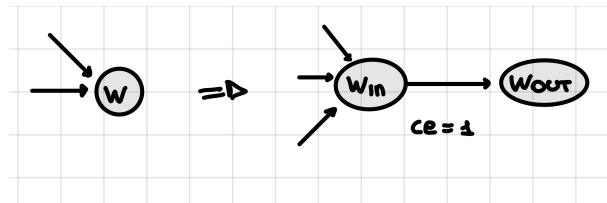
($\Leftarrow$) If the max-flow equals $|X|$, all edges (s, u) for $u \in X$ are saturated, so one unit of flow leaves each populated node. By flow conservation and the fact that all capacities are 1, each unit of flow travels along a distinct path from some $u \in X$ to some $v \in S$. Since each edge carries at most 1 unit, these paths are edge-disjoint. $\square$

This construction has $O(|V|)$ nodes and $O(|E|)$ edges, so it runs in polynomial time.

Part (ii): Node-Disjoint Evacuation Routes

To enforce that paths do not share any intermediate node, we use the **node-splitting technique**.

Construction of G'' . For **every** node $w \in V$, we replace it with two nodes w_{in} and w_{out} , connected by a directed edge $(w_{\text{in}}, w_{\text{out}})$. The capacity of this internal edge depends on the role of w :



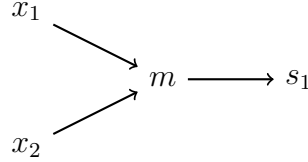
The remaining edges are built as follows:

- **Internal edges:** Every original edge $(u, v) \in E$ becomes an edge $(u_{\text{out}}, v_{\text{in}})$ with capacity 1.
- **Source edges:** Add a super-source s and, for each $u \in X$, add edge (s, u_{in}) with capacity 1.

- **Sink edges:** Add a super-sink t and, for each $v \in S$, add edge (v_{out}, t) with capacity $|X|$.

Part (iii): An Example Where (a) is Yes but (b) is No

Consider the following graph with $X = \{x_1, x_2\}$, $S = \{s_1\}$, and a single intermediate node m :



Formally: $V = \{x_1, x_2, m, s_1\}$, $E = \{(x_1, m), (x_2, m), (m, s_1)\}$, $X = \{x_1, x_2\}$, $S = \{s_1\}$.

- **Part (a) answer: Yes.** The two paths $x_1 \rightarrow m \rightarrow s_1$ and $x_2 \rightarrow m \rightarrow s_1$ share the node m but share *no edge*. So edge-disjoint evacuation routes exist.
- **Part (b) answer: No.** Both paths must pass through m to reach s_1 . Since m is the only intermediate node, there is no way to route both x_1 and x_2 to S without sharing m . The max-flow in G'' is 1 (the bottleneck is the single edge $m_{\text{in}} \rightarrow m_{\text{out}}$ with capacity 1), which is less than $|X| = 2$.

Problem 3 :Free-Standing Subset

We are given a set of n countries that are engaged in trade with one another. For each country i , we have the value s_i of its budget surplus; this number may be positive or negative, with a negative number indicating a deficit. For each pair of countries i, j , we have the total value e_{ij} of all exports from i to j ; this number is always nonnegative. We say that a subset S of the countries is free-standing if the sum of the budget surpluses of the countries in S , minus the total value of all exports from countries in S to countries not in S , is nonnegative. Give a polynomial-time algorithm that takes this data for a set of n countries and decides whether it contains a nonempty free-standing subset that is not equal to the full set.

Given n countries with budget surpluses s_i and export values $e_{ij} \geq 0$, a subset S is *free-standing* if

$$\sum_{i \in S} s_i - \sum_{i \in S, j \notin S} e_{ij} \geq 0.$$

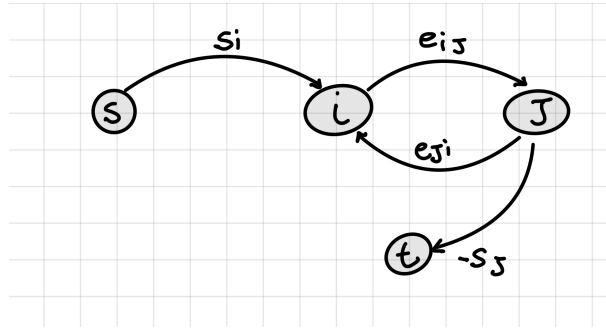
Give a polynomial-time algorithm to decide whether a nonempty free-standing subset $S \subsetneq \{1, \dots, n\}$ exists.

Key Observation. We rewrite the free-standing condition. Let $\bar{S} = \{1, \dots, n\} \setminus S$. A subset S is free-standing iff:

$$\sum_{i \in S} s_i - \sum_{\substack{i \in S \\ j \in \bar{S}}} e_{ij} \geq 0 \iff \sum_{\substack{i \in S \\ j \in \bar{S}}} e_{ij} \leq \sum_{i \in S} s_i.$$

Construction of the Flow Network G' . We reduce the problem to a **minimum cut** problem. Build a directed graph $G' = (V', E')$ as follows:

- **Nodes:** One node per country, plus a super-source s and a super-sink t . So $V' = \{1, \dots, n\} \cup \{s, t\}$.
- **Edges between countries:** For each pair $i \neq j$, add edge (i, j) with capacity $c(i, j) = e_{ij}$.
- **Source edges:** For each country i with $s_i > 0$, add edge (s, i) with capacity $c(s, i) = s_i$.
- **Sink edges:** For each country i with $s_i < 0$, add edge (i, t) with capacity $c(i, t) = -s_i$.



Algorithm. Fix any country k . For each other country $\ell \neq k$, compute the minimum s - t cut in G' with k forced on the source side and ℓ forced on the sink side. A nonempty free-standing proper subset exists if and only if the minimum such cut has capacity less than $\sum_{i: s_i > 0} s_i$.

Alternatively, run $n - 1$ min-cut computations (fixing one country on each side) and check whether any of them corresponds to a free-standing subset.

Correctness. Consider any cut (A, B) with $s \in A$ and $t \in B$. Let $S = A \setminus \{s\}$ be the set of countries on the source side. The capacity of this cut is:

$$\text{cap}(A, B) = \sum_{\substack{i \in S \\ j \in \bar{S}}} e_{ij} + \sum_{\substack{i \in S \\ s_i < 0}} (-s_i) + \sum_{\substack{j \in \bar{S} \\ s_j > 0}} s_j.$$

One can verify that S is free-standing if and only if this cut capacity is at most $\sum_{i: s_i > 0} s_i$, which is the capacity of the trivial cut $(\{s\}, V' \setminus \{s\})$. Therefore:

Claim 3. *Minimizing $\text{cap}(A', V \setminus A')$ over all non-trivial cuts is equivalent to maximizing $c(A)$, where:*

$$\text{cap}(A', V \setminus A') = P + \sum_{\substack{i \in A \\ j \in A}} e_{ij} - \sum_{i \in A} s_i.$$

In particular, a free-standing subset exists if and only if

$$\text{cap}(A', V \setminus A') \leq P.$$

Chapter 7

NP and Computational Intractability

7.1 Algorithm Design Patterns and Anti-Patterns

Some problems can be solved with well-known **design patterns**:

- **Greedy** – e.g., interval scheduling in $O(n \log n)$.
- **Divide-and-conquer** – e.g., FFT in $O(n \log n)$.
- **Dynamic programming** – e.g., edit distance in $O(n^2)$.
- **Duality** – e.g., bipartite matching in $O(n^3)$.
- **Reductions**, local search, randomization.

Other problems seem to have **no efficient algorithm**. These are called **anti-patterns**:

- **NP-completeness** – an $O(n^k)$ algorithm is very unlikely.
- **PSPACE-completeness** – even an $O(n^k)$ certifier is unlikely.
- **Undecidability** – no algorithm exists at all.

7.2 Polynomial-Time Reductions (Section 8.1)

7.2.1 Classifying Problems

Key question: Which problems can we solve in practice?

Yes	Probably no
Shortest path	Longest path
Matching	3D-matching
Min cut	Max cut
2-SAT	3-SAT
Planar 4-color	Planar 3-color
Bipartite vertex cover	Vertex cover
Primality testing	Factoring

7.2.2 Polynomial-Time Reduction

Definition 24. *Polynomial-Time Reduction* Problem X **polynomially reduces** to problem Y , written $X \leq_P Y$, if any instance of X can be solved using:

- a polynomial number of standard computational steps, plus
- a polynomial number of calls to an **oracle** that solves Y .

Note: Instances of Y must have polynomial size (we pay to write them down).

Why reductions matter:

- **Design algorithms:** If $X \leq_P Y$ and Y can be solved in poly-time, then X can too.
- **Prove intractability:** If $X \leq_P Y$ and X has no poly-time algorithm, then Y has no poly-time algorithm either.
- **Prove equivalence:** If $X \leq_P Y$ and $Y \leq_P X$, we write $X \equiv_P Y$.

7.2.3 Three Basic Reduction Strategies

1. Reduction by **simple equivalence**.
2. Reduction from **special case** to **general case**.
3. Reduction by **encoding with gadgets**.

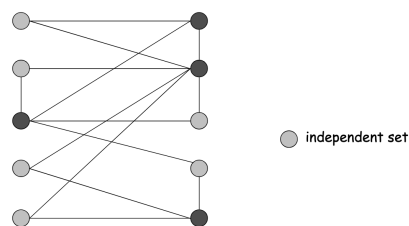
7.2.4 Strategy 1 — Simple Equivalence

Problem: Independent Set

Definition 25. *Independent Set* Given a graph $G = (V, E)$ and an integer k : does there exist a subset $S \subseteq V$ with $|S| \geq k$ such that **no two vertices in S share an edge**?

Independent Set Consider a graph with 8 nodes.

Ex. Is there an independent set of size ≥ 6 ? Yes.
Ex. Is there an independent set of size ≥ 7 ? No.

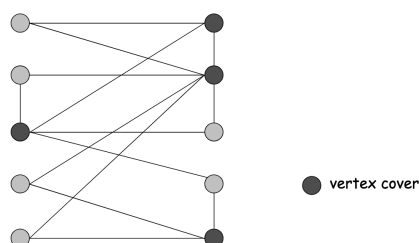


Problem: Vertex Cover

Definition 26. *Vertex Cover* Given a graph $G = (V, E)$ and an integer k : does there exist a subset $S \subseteq V$ with $|S| \leq k$ such that **every edge has at least one endpoint in S** ?

Vertex Cover On the same graph:

Ex. Is there a vertex cover of size ≤ 4 ? Yes.
Ex. Is there a vertex cover of size ≤ 3 ? No.



min numero di nodi che coprono tutti gli archi

Claim 4. $VERTEX-COVER \equiv_P INDEPENDENT-SET$ S is an independent set $\iff V \setminus S$ is a vertex cover.

Proof ($\Rightarrow$): Let S be an independent set. Take any edge (u, v) . Since S is independent, $u \notin S$ or $v \notin S$, so $u \in V \setminus S$ or $v \in V \setminus S$. Thus $V \setminus S$ covers every edge.

Proof ($\Leftarrow$): Let $V \setminus S$ be a vertex cover. Take any two nodes $u, v \in S$. The edge (u, v) cannot exist because $V \setminus S$ would need to cover it, but neither u nor v is in $V \setminus S$. So no two nodes in S are connected $\Rightarrow S$ is independent. $\square$

7.2.5 Strategy 2 — Special Case to General Case

Definition 27. *Set Cover* Given a universe U of elements, a collection of subsets $S_1, S_2, \dots, S_m \subseteq U$, and an integer k : does there exist a collection of $\leq k$ sets whose **union equals U** ?

Set Cover

$$U = \{1, 2, 3, 4, 5, 6, 7\}, \quad k = 2$$

$$S_1 = \{3, 7\}, \quad S_2 = \{3, 4, 5, 6\}, \quad S_3 = \{1\}, \quad S_4 = \{2, 4\}, \quad S_5 = \{5\}, \quad S_6 = \{1, 2, 6, 7\}$$

Answer: **Yes**, choose S_2 and S_6 : their union is $\{1, 2, 3, 4, 5, 6, 7\} = U$.

Real-world application: You have m software packages. Each gives a set S_i of capabilities. Can you cover all n required capabilities using $\leq k$ packages?

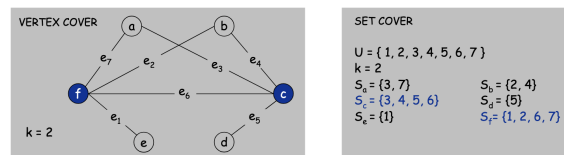
Claim 5. $VERTEX-COVER \leq_P SET-COVER$ **Construction:** Given a $VERTEX-COVER$ instance $\langle G = (V, E), k \rangle$, create a $SET-COVER$ instance:

$$k' = k, \quad U = E, \quad S_v = \{e \in E : e \text{ is incident to vertex } v\}$$

Why it works: A set of vertices covers all edges iff the corresponding sets cover all elements. A vertex cover of size $\leq k$ exists $\iff$ a set cover of size $\leq k$ exists.

Example: A graph with vertices $\{a, b, c, d, e, f\}$ and edges $\{e_1, \dots, e_7\}$.

Taking $k = 2$, vertex cover $\{b, e\}$ corresponds to sets $S_b = \{e_1, e_2, e_3, e_4\}$, $S_e = \{e_4, e_5, e_6, e_7\}$.



Strategy 3 — Encoding with Gadgets

Definition 28. SAT and $3-SAT$

- A **literal** is a Boolean variable x_i or its negation $\overline{x_i}$.
- A **clause** is a disjunction (OR) of literals: $C_j = x_1 \vee \overline{x_2} \vee x_3$.
- A formula in **Conjunctive Normal Form (CNF)** is a conjunction (AND) of clauses:

$$\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$$

- **SAT:** Given a CNF formula Φ , does it have a satisfying truth assignment?
- **3-SAT:** SAT where every clause has **exactly 3 literals**.

3-SAT

3-SAT: SAT where each clause contains exactly 3 literals.

each corresponds to a different variable

Ex: $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$
Yes: $x_1 = \text{true}, x_2 = \text{true}, x_3 = \text{false}$.

Claim 6. $3-SAT \leq_P INDEPENDENT-SET$ **Construction:** Given a 3-SAT instance Φ with k clauses, build graph G :

1. For each clause, create **3 vertices** (one per literal). Connect them in a **triangle**.
2. Connect each literal vertex to every vertex representing its **negation** in other clauses.

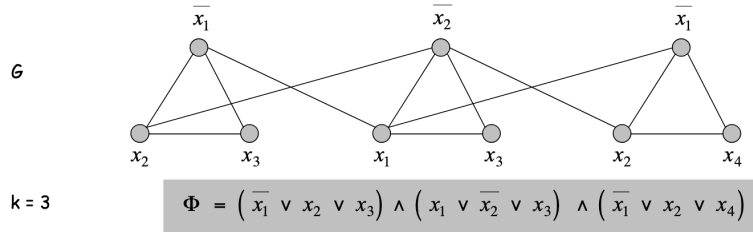
Example:

$$\Phi = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4)$$

$k = 3$, so we need an independent set of size 3.

The graph G has $3 \times 3 = 9$ vertices.

Triangle edges (within each clause) + conflict edges (e.g., x_1 in clause 1 is connected to $\bar{x}_1$ in clause 2).



scegliamo questo tipo di costruzione perchè vogliamo un independent set di dimensione k , quindi se ogni clausola è rappresentata da un triangolo, sono costretta a scegliere un vertice quindi un literal per ogni clausola. Inoltre, i conflict edges mi impediscono di scegliere due literal che si contraddicono (es. x_1 e $\bar{x}_1$).

Proof: G has an independent set of size $k \iff \Phi$ is satisfiable.

- ($\Rightarrow$) An independent set of size k picks exactly one vertex per triangle (one per clause). Triangles prevent choosing two from the same clause. Conflict edges prevent contradictory assignments. Setting those literals to **true** satisfies every clause consistently.
- ($\Leftarrow$) Given a satisfying assignment, pick one **true** literal per clause. These form an independent set of size k (no two are in the same triangle, and no two conflict).

in questo caso il set= x_3, x_1, x_4 è un independent set di dimensione 3, quindi Φ è soddisfacibile (es. $x_1 = x_2 = x_3 = \text{true}$, x_4 arbitrary).

7.2.6 Transitivity of Reductions

Theorem 27. Transitivity If $X \leq_P Y$ and $Y \leq_P Z$, then $X \leq_P Z$.

This means we can **chain** reductions. A key example:

$$3\text{-SAT} \leq_P \text{INDEPENDENT-SET} \leq_P \text{VERTEX-COVER} \leq_P \text{SET-COVER}$$

7.3 Definition of NP (Section 8.3)

7.3.1 Decision Problems

A **decision problem** X is a set of strings. An algorithm A **solves** X if $A(s) = \text{yes} \iff s \in X$.

Polynomial time: A runs in poly-time if for every input s , $A(s)$ terminates in at most $p(|s|)$ steps, where p is a polynomial and $|s|$ is the length of s .

7.3.2 The Class P

Definition 29. Class P is the set of all decision problems solvable by a **polynomial-time algorithm**.

Problems in P

Problem	Question	Algorithm
MULTIPLE	Is x a multiple of y ?	Grade school division
RELPRIME	Are x and y relatively prime?	Euclid's algorithm
PRIMES	Is x prime?	AKS algorithm
EDIT-DISTANCE	Is the edit distance between x and y less than 5?	Dynamic programming
LSOLVE	Does $Ax = b$ have a solution?	Gauss-Edmonds elimination

PRIMES example:

- Input: $s = 53$ (binary string of length $|s| = 6$ bits).
- Output: **yes**, 53 is prime.
- Time: $O(|s|^8)$ – polynomial in the **length** of the input, not its value.

7.3.3 The Class NP

Intuition: Instead of *finding* a solution, can we *verify* one quickly?

Definition 30. Certifier Algorithm $C(s, t)$ is a **certifier** for problem X if for every string s :

$$s \in X \iff \exists t \text{ such that } C(s, t) = \text{yes}$$

The string t is called the **certificate** (or witness).

Definition 31. Class NP is the set of decision problems for which there exists a **poly-time certifier** $C(s, t)$ with $|t| \leq p(|s|)$ for some polynomial p .

NP stands for Nondeterministic Polynomial-time.

COMPOSITES is in NP Problem: Given an integer s , is s composite (not prime)?

Certificate: A nontrivial factor t of s (i.e., $1 < t < s$ and $t \mid s$).

Certifier:

```
boolean C(s, t) {
    if (t <= 1 or t >= s) return false;
    if (s mod t == 0)      return true;
    else                  return false;
}
```

Concrete instance:

- $s = 437,669$ (Is it composite?)
- Certificate: $t = 541$ (or $t = 809$).

- Verify: $437,669 = 541 \times 809$. ✓
- Conclusion: **COMPOSITES is in NP**.

Why $|t| \leq |s|$? Any factor of s is at most s , so it has at most the same number of bits.

SAT is in NP Problem: Given a CNF formula Φ , does it have a satisfying assignment?

Certificate: An assignment of truth values to all n Boolean variables.

Certifier: Check that each clause contains at least one **true** literal.

Concrete instance:

$$\Phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_4) \wedge (x_1 \vee \overline{x_3} \vee \overline{x_4})$$

Certificate: $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 1$.

Check clause 1: $1 \vee 0 \vee 1 = 1$. ✓ Check clause 2: $0 \vee 1 \vee 1 = 1$. ✓ (all clauses satisfied)

Conclusion: SAT is in NP.

HAM-CYCLE is in NP Problem: Given an undirected graph $G = (V, E)$, does there exist a **Hamiltonian cycle** – a simple cycle that visits every node exactly once?

Certificate: A permutation of all n nodes.

Certifier:

1. Check that the permutation contains each node in V exactly once.
2. Check that there is an edge between each pair of adjacent nodes in the permutation.
3. Check that there is an edge between the last and first node (to close the cycle).

Conclusion: HAM-CYCLE is in NP. (The certifier runs in $O(n)$ – polynomial.)

7.3.4 P, NP, and EXP

Definition 32. Class **EXP** is the set of decision problems solvable by an **exponential-time** algorithm.

Theorem 28. Inclusion Chain

$$P \subseteq NP \subseteq EXP$$

Proof of $P \subseteq NP$: If algorithm $A(s)$ solves X in poly-time, use empty certificate ε and define $C(s, \varepsilon) = A(s)$. So $X \in NP$.

Proof of $NP \subseteq EXP$: For $X \in NP$, there exists a poly-time certifier $C(s, t)$.

To solve input s : run $C(s, t)$ on all strings t with $|t| \leq p(|s|)$.

There are at most $2^{p(|s|)}$ such strings – exponentially many, but finite.

Return **yes** if any call returns **yes**. This is an exponential algorithm. $\square$

7.3.5 The P vs. NP Question

The biggest open question in computer science: Does $P = NP$?

- If $P = NP$: efficient algorithms exist for 3-SAT, TSP, 3-COLOR, FACTOR, ...
This would **break RSA cryptography** and potentially collapse the economy.
- If $P \neq NP$: no efficient algorithm exists for NP-complete problems.
- **Consensus:** most researchers believe $P \neq NP$. (Probably no.)

If $P \neq NP$	If $P = NP$
$EXP \supset NP \supset P$	$EXP \supset P = NP$
(all three classes are distinct)	(P and NP collapse together)

7.4 NP-Completeness (Section 8.4)

7.4.1 Two Types of Reduction

- **Cook reduction** ($\leq_P$): solve X using a polynomial number of oracle calls to Y .
- **Karp transformation**: given input x to X , construct y (in poly-time) such that $x \in X \iff y \in Y$. (Only **one oracle call**, at the very end.)

Almost all reductions seen in practice are **Karp transformations**. The open question is whether these two definitions are actually equivalent.

7.4.2 NP-Complete Problems

Definition 33. *NP-Complete* A problem Y is **NP-complete** if:

1. $Y \in NP$.
2. For every $X \in NP$: $X \leq_P Y$. (“ Y può risolvere qualsiasi problema in NP ”.)

Theorem 29. Central Theorem Suppose Y is NP-complete. Then Y is solvable in poly-time $\iff P = NP$.

Proof ($\Leftarrow$): If $P = NP$, then since $Y \in NP$, $Y \in P$, so Y is solvable in poly-time.

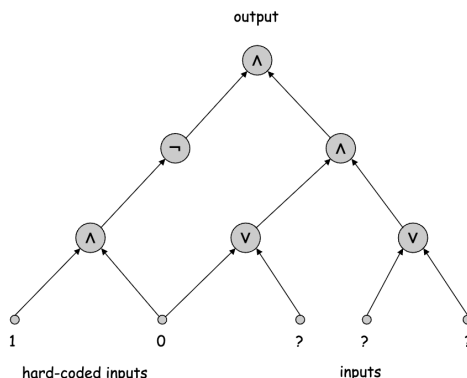
Proof ($\Rightarrow$): Suppose Y is solvable in poly-time. Let X be any problem in NP .

Since $X \leq_P Y$, we can solve X in poly-time $\Rightarrow NP \subseteq P$.

We already know $P \subseteq NP$, so $P = NP$. $\square$

7.4.3 Circuit Satisfiability — The First NP-Complete Problem

Definition 34. CIRCUIT-SAT Given a combinational circuit built from AND, OR, and NOT gates, where some inputs are **fixed** (hard-coded 0 or 1) and some are **free** (the inputs we can choose): can we set the free inputs so the circuit **output is 1**?



Question: Can we set $x_2, x_3, x_4 \in \{0, 1\}$ so the output is 1?

Example answer: $x_2 = 1, x_3 = 0, x_4 = 1$ produces output 1. ✓

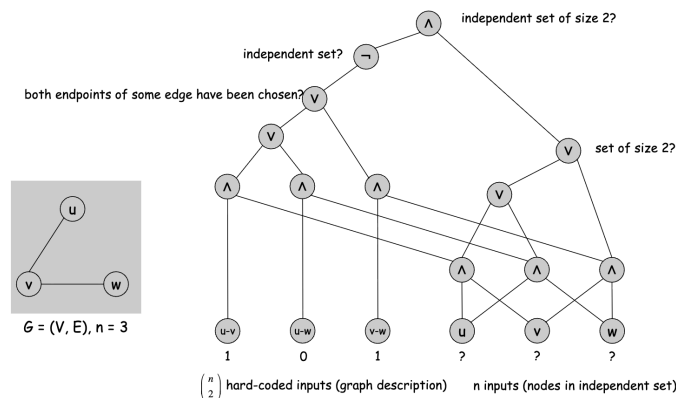
Theorem 30. Cook-Levin Theorem (1971/1973) **CIRCUIT-SAT is NP-complete.**

Proof sketch:

- **CIRCUIT-SAT** $\in$ NP: Certificate = a setting of the free inputs. Certifier = simulate the circuit ($O(n)$ time).
- Every NP problem $\leq_P$ CIRCUIT-SAT: For any $X \in$ NP with certifier $C(s, t)$:
 1. View $C(s, t)$ as an algorithm on $|s| + p(|s|)$ input bits (fixed s , free t).
 2. Since C runs in poly-time, it can be converted into a poly-size circuit K .
 3. Hard-code the first $|s|$ bits with the input s .
 4. The remaining $p(|s|)$ bits represent the certificate t (free inputs).
 5. K is satisfiable $\iff C(s, t) = \text{yes}$ for some $t \iff s \in X$.

Concrete example: Encode the INDEPENDENT-SET certifier as a circuit.

For graph $G = (V, E)$ with $n = 3$ (nodes u, v, w) and $k = 2$:



- The circuit has $n = 3$ free inputs (which nodes to include).
- Hard-coded inputs encode the graph structure (which edges exist).
- The circuit outputs 1 iff the chosen nodes form an independent set of size 2.

7.4.4 Recipe to Prove NP-Completeness

Once CIRCUIT-SAT is established, **other NP-complete problems fall like dominoes.**

1. **Show $Y \in$ NP:** Describe a certificate and a poly-time certifier.
2. **Choose a known NP-complete problem X** (e.g., CIRCUIT-SAT, 3-SAT).
3. **Prove $X \leq_P Y$:** Construct a poly-time transformation.

Justification: If $W \in$ NP, then $W \leq_P X \leq_P Y$ (by transitivity), so Y is NP-complete.

7.4.5 3-SAT is NP-Complete

Theorem 31. *3-SAT is NP-complete* **Proof:** $3\text{-SAT} \in NP$ (certificate: truth assignment). It suffices to show $\text{CIRCUIT-SAT} \leq_P 3\text{-SAT}$.

Construction: Given a circuit K , create a 3-SAT variable x_i for each element (gate or wire) i . Add clauses to force correct computation:

Gate type	Constraint	Clauses added
NOT gate	$x_2 = \neg x_3$	$(\overline{x_2} \vee \overline{x_3}) \wedge (x_2 \vee x_3)$ (2 clauses)
OR gate	$x_1 = x_4 \vee x_5$	3 clauses
AND gate	$x_0 = x_1 \wedge x_2$	3 clauses
Hard-coded 0	$x_5 = 0$	$(\overline{x_5})$ (1 clause)
Output = 1	$x_0 = 1$	(x_0) (1 clause)

Short clauses (of length 1 or 2) are padded to length exactly 3 using new dummy variables.

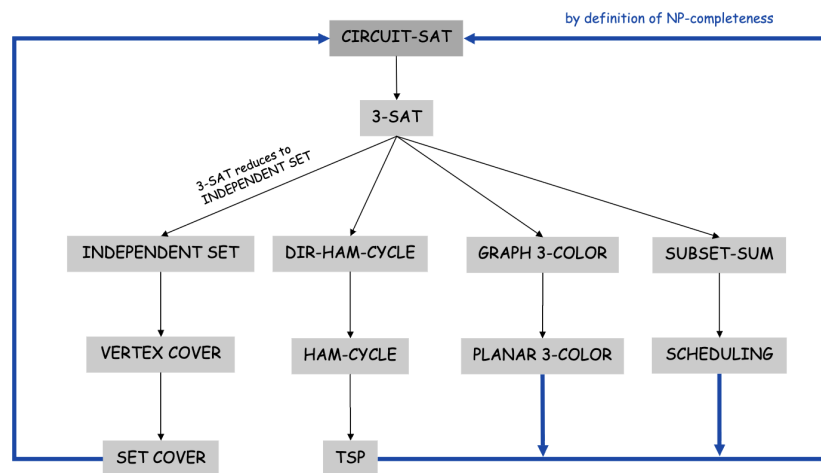
Result: The formula is satisfiable $\iff$ the circuit is satisfiable. $\square$

7.4.6 The Domino Chain of NP-Complete Problems

All problems below are NP-complete and reduce to one another:

$\text{CIRCUIT-SAT} \leq_P 3\text{-SAT} \leq_P \text{INDEPENDENT-SET} \leq_P \text{VERTEX-COVER} \leq_P \text{SET-COVER}$

Also NP-complete: DIR-HAM-CYCLE, HAM-CYCLE, TSP, GRAPH 3-COLOR, PLANAR 3-COLOR, SUBSET-SUM, SCHEDULING, ...



22

Most NP problems are either known to be in P or NP-complete. Notable exceptions (in between, not known to be either): FACTORING, graph isomorphism, Nash equilibrium.

MANCAAAAA

Chapter 8

APPROXIMATION ALGORITHMS(11)

8.1 Coping with NP-completeness

Some problems are NP-hard, which means we believe no polynomial-time algorithm can solve them exactly. So what do we do in practice?

We have to give something up. There are three things we would like an algorithm to have:

1. It works on **any** input (not just easy special cases).
2. It finds the **exact best** solution (optimal).
3. It runs in **polynomial time** (fast).

Since we cannot have all three for NP-hard problems, we give up on **point 2**: we accept a solution that is *not perfect*, but is *provably close* to the best possible.

Definition 35. ρ -Approximation Algorithm A ρ -**approximation algorithm** is an algorithm that always:

- runs in polynomial time,
- gives a solution whose cost is at most ρ times the true optimal cost (for minimization problems).

If $\rho = 2$, the algorithm is a **2-approximation**: it always finds a solution at most **twice as bad** as the best possible.

The key challenge: To prove the approximation ratio ρ , we need to compare our solution to the optimal. But we do not know the optimal! The trick is to use **lower bounds** — quantities that the optimal must be at least, even if we cannot compute the optimal directly.

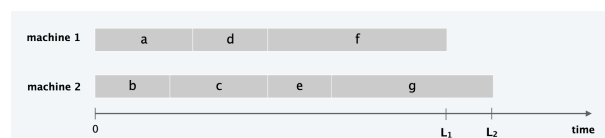
8.2 Load Balancing

8.2.1 The Problem

Imagine you have m machines and n jobs. Each job j takes t_j units of time and must run on exactly one machine. You want to split the jobs across machines so that no machine is overloaded.

Formally:

- The **load** of machine i is $L_i = \sum_{j \in J(i)} t_j$ (sum of times of all jobs on machine i).
- The **makespan** is $L = \max_i L_i$ (the time when the last machine finishes).
- **Goal:** minimise the makespan.



This problem is NP-hard even with only 2 machines.

questo problema è riconducibile al number partitioning problem, che è NP-hard. Se avessimo un algoritmo polinomiale per il load balancing, potremmo usarlo per risolvere il number partitioning: basta mettere i numeri come job e le due partizioni come macchine. Se il load balancing restituisce un makespan di 0, allora i numeri possono essere divisi in due partizioni con somma uguale (soluzione al number partitioning). Se restituisce un makespan maggiore di 0, allora non esiste una partizione perfetta.

8.2.2 Algorithm 1: List Scheduling (Greedy)

```
List-Scheduling( $m, n, t_1, t_2, \dots, t_n$ ) {  
  for  $i = 1$  to  $m$  {  
     $L_i \leftarrow 0$             $\leftarrow$  load on machine  $i$   
     $J(i) \leftarrow \emptyset$    $\leftarrow$  jobs assigned to machine  $i$   
  }  
  
  for  $j = 1$  to  $n$  {  
     $i = \operatorname{argmin}_k L_k$      $\leftarrow$  machine  $i$  has smallest load  
     $J(i) \leftarrow J(i) \cup \{j\}$   $\leftarrow$  assign job  $j$  to machine  $i$   
     $L_i \leftarrow L_i + t_j$      $\leftarrow$  update load of machine  $i$   
  }  
  return  $J(1), \dots, J(m)$   
}
```

Implementation. $O(n \log m)$ using a priority queue.

Idea: Go through the jobs one by one, and always put the next job on the machine that currently has the least work.

List Scheduling 3 machines, jobs with times $[3, 3, 2, 2, 10]$.

- Job 1 ($t = 3$) $\rightarrow$ Machine 1. Loads: $[3, 0, 0]$.
- Job 2 ($t = 3$) $\rightarrow$ Machine 2 (smallest load). Loads: $[3, 3, 0]$.
- Job 3 ($t = 2$) $\rightarrow$ Machine 3. Loads: $[3, 3, 2]$.
- Job 4 ($t = 2$) $\rightarrow$ Machine 3. Loads: $[3, 3, 4]$.
- Job 5 ($t = 10$) $\rightarrow$ Machine 1 or 2. Loads: $[13, 3, 4]$.

Makespan = 13. (The optimal is also 10, so this is close but not perfect.)

8.2.3 Lower Bounds on the Optimal L^*

We need two facts about the optimal makespan L^* that we can prove without knowing L^* exactly.

Lemma 10. *Two Lower Bounds*

1. $L^* \geq \max_j t_j$ (the optimal cannot be less than the longest single job, because some machine must run it).
2. $L^* \geq \frac{1}{m} \sum_j t_j$ (the optimal cannot be less than the average load: the total work is $\sum t_j$, split among m machines, so at least one machine must do at least the average share).

8.2.4 Analysis: List Scheduling is a 2-Approximation

Theorem 32. *List Scheduling $\leq 2L^*$ (l'algoritmo è una 2-approssimazione, cioè trova al massimo una soluzione che è doppia alla soluzione ottima)*

Proof (step by step):

1. Let i be the **bottleneck machine** (the machine with the highest load $L_i = L$).
2. Let j be the **last job assigned to machine i** .
3. At the moment job j was assigned, machine i had the **smallest load** of all machines (that is why the algorithm chose it). So its load at that moment was $L_i - t_j$, and:

$$L_i - t_j \leq L_k \quad \text{for every other machine } k.$$

4. Summing this inequality over all m machines and dividing by m :

$$L_i - t_j \leq \frac{1}{m} \sum_{k=1}^m L_k = \frac{1}{m} \sum_j t_j \leq L^* \quad (\text{by Lemma, part 2})$$

Se M_i era la più scarica, il suo carico era minore al carico medio di tutte le macchine.

5. Now we compute the final load of the bottleneck machine:

$$L = L_i = \underbrace{(L_i - t_j)}_{\leq L^*} + \underbrace{t_j}_{\leq L^*} \leq 2L^* \quad (\text{using Lemma part 1 for } t_j \leq L^*)$$

Conclusion: $L \leq 2L^*$. $\square$

The 2-approximation is tight (nearly) Take $m = 10$ machines.

Create $m(m - 1) = 90$ jobs of length 1, and 1 job of length $m = 10$.

List scheduling assigns all 90 short jobs first: each machine gets $m - 1 = 9$ short jobs (load 9). Then the big job goes on machine 1: its load becomes $9 + 10 = 19$. **Makespan = 19.**

Optimal: put the big job alone on one machine (load 10), distribute the 90 short jobs among the 9 others (10 each). **Optimal makespan = 10.**

Ratio: $\frac{19}{10} \approx 1.9 \approx 2$.

8.2.5 Algorithm 2: Longest Processing Time (LPT)

```

LPT-List-Scheduling( $m, n, t_1, t_2, \dots, t_n$ ) {
  Sort jobs so that  $t_1 \geq t_2 \geq \dots \geq t_n$ 

  for  $i = 1$  to  $m$  {
     $L_i \leftarrow 0$            ← load on machine  $i$ 
     $J(i) \leftarrow \emptyset$  ← jobs assigned to machine  $i$ 
  }

  for  $j = 1$  to  $n$  {
     $i = \operatorname{argmin}_k L_k$       ← machine  $i$  has smallest load
     $J(i) \leftarrow J(i) \cup \{j\}$  ← assign job  $j$  to machine  $i$ 
     $L_i \leftarrow L_i + t_j$     ← update load of machine  $i$ 
  }
  return  $J(1), \dots, J(m)$ 
}

```

Improvement idea: Before running list scheduling, **sort the jobs from longest to shortest**. This prevents the worst case above (big job coming last).

Theorem 33. *LPT is a $\frac{3}{2}$ -Approximation*

Key extra lemma: *If there are more than m jobs, then $L^* \geq 2t_{m+1}$.*

Why? The first $m + 1$ jobs (in sorted order) are all at least as long as t_{m+1} . With $m + 1$ jobs and m machines, by the pigeonhole principle, at least one machine gets two of them. So $L^ \geq 2t_{m+1}$.*

Proof sketch: Following the same logic as before, let j be the last job on the bottleneck machine. After sorting, $j \geq m + 1$ (it is not one of the first m jobs), so:

$$t_j \leq t_{m+1} \leq \frac{1}{2}L^*$$

Therefore:

$$L = (L_i - t_j) + t_j \leq L^* + \frac{1}{2}L^* = \frac{3}{2}L^*$$

□

8.3 Center Selection

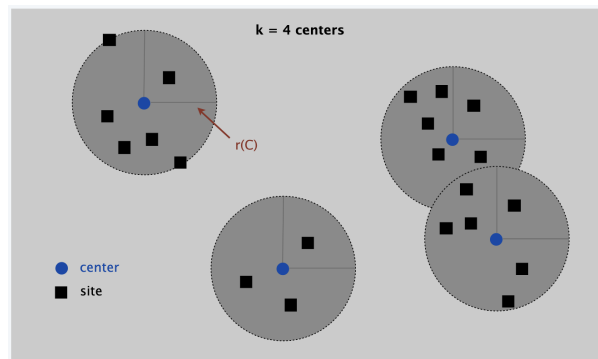
8.3.1 The Problem

You have n sites (think: towns) and you need to build k facilities (think: hospitals). You want to place k centers so that every site is **as close as possible** to some center.

Formally, the **covering radius** is:

$$r(C) = \max_i \text{dist}(s_i, C) = \max_i \min_{c \in C} \text{dist}(s_i, c)$$

Goal: Choose k centers C to minimise $r(C)$.



The distance satisfies three natural properties: $\text{dist}(x, x) = 0$ (identity), $\text{dist}(x, y) = \text{dist}(y, x)$ (symmetry), and $\text{dist}(x, y) \leq \text{dist}(x, z) + \text{dist}(z, y)$ (triangle inequality).

8.3.2 Wrong Greedy: A False Start

Bad idea: Place the first center at the globally best single location, then greedily reduce the radius.

Problem: This can give an arbitrarily bad answer. The first center might “attract” both clusters, and the second center is placed far from where it is really needed.

8.3.3 Correct Greedy Algorithm

Right idea: Always place the next center at the **site farthest from any existing center**.

GREEDY-CENTER-SELECTION ($k, n, s_1, s_2, \dots, s_n$)

$C \leftarrow \emptyset$.

REPEAT k times

 Select a site s_i with maximum distance $\text{dist}(s_i, C)$.

$C \leftarrow C \cup s_i$.

RETURN C .

↑
site farthest
from any center

Key property after termination: All k centers in C are pairwise at distance $\geq r(C)$ from each other.

Why? Because each new center was chosen as the farthest site at the time it was added. Once it is a center, no later center can be closer than $r(C)$ to it.

Theorem 34. *Center Selection Greedy is a 2-Approximation Let C^* be the optimal set of k centers with radius $r^* = r(C^*)$.*

Let C be the greedy solution with radius $r = r(C)$.

Then $r \leq 2r^$.*

Proof by contradiction:

Assume $r^* < \frac{1}{2}r$, i.e., the optimal radius is less than half our radius.

1. Draw a ball of radius $\frac{1}{2}r$ around each of our k greedy centers $c_i \in C$.
2. Our centers are all at distance $\geq r$ from each other, so these balls do not overlap.
3. The optimal solution C^* has k centers. Since there are k non-overlapping balls and k optimal centers, by the pigeonhole principle, each ball contains exactly one optimal center c_i^* .
4. Now take any site s . It has some closest optimal center c_i^* at distance $\leq r^*$.
By the triangle inequality:

$$\text{dist}(s, c_i) \leq \text{dist}(s, c_i^*) + \text{dist}(c_i^*, c_i) \leq r^* + r^* = 2r^*$$

(since both s and c_i are within r^* of c_i^* , as the ball has radius $\frac{1}{2}r < r^* \dots$ wait, let's be precise:)

We assumed $r^* < \frac{1}{2}r$, so $\text{dist}(c_i^*, c_i) \leq \frac{1}{2}r < r^*$.

Therefore $\text{dist}(s, c_i) \leq r^* + r^* = 2r^*$.

5. This means $r(C) \leq 2r^*$, which **contradicts** our assumption. $\square$

Center selection in practice Sites: cities at positions 1, 5, 6, 10 on a line. $k = 2$ centers.

Greedy: Start at city 1. Farthest site is city 10 $\rightarrow$ add it. Done: centers = $\{1, 10\}$.

Radius: worst site is 5 or 6. $\text{dist}(5, \{1, 10\}) = \min(4, 5) = 4$. So $r = 4$.

Optimal: centers = $\{3.5, 8\}$ (non-sites!), giving radius = 2.5.

Note: the greedy is constrained to place centers *on sites* yet gives $r = 4 \leq 2 \times 2.5 = 5$. The factor-2 bound holds.

Is the 2-approximation the best possible?

Yes (unless $P = NP$). A $(2 - \varepsilon)$ -approximation would let us solve DOMINATING-SET in polynomial time:

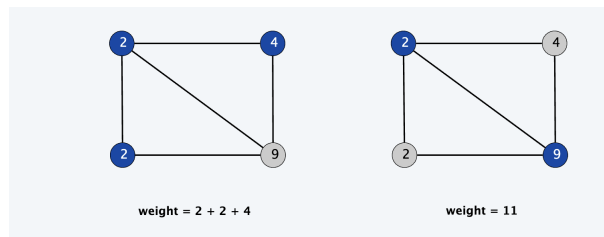
- Build a CENTER-SELECTION instance from graph G : set $\text{dist}(u, v) = 1$ if edge, $\text{dist}(u, v) = 2$ if no edge.
- G has a dominating set of size $k \iff$ there are k centers with radius $r^* = 1$.
- A $(2 - \varepsilon)$ -approximation would find radius $r = 1$ (since it cannot use edges of distance 2), solving DOMINATING-SET in poly-time.

8.4 Pricing Method: Vertex Cover

8.4.1 Weighted Vertex Cover

Recall that a **vertex cover** is a set of vertices S such that every edge has at least one endpoint in S . Now each vertex i has a weight $w_i \geq 0$, and we want the minimum total weight cover.

Weighted Vertex Cover



The pricing method will find an approximation of the minimum weight cover.

8.4.2 The Pricing Idea

Think of each edge as a **client** that must pay to use two vertices. Edge $e = (i, j)$ pays a price $p_e \geq 0$, and this price is split between vertex i and vertex j .

Fairness rule: No vertex should be “overcharged”. The total price paid by all edges incident to vertex i must be $\leq w_i$:

$$\sum_{e \ni i} p_e \leq w_i \quad \text{for all vertices } i$$

A vertex is called **tight** when this inequality becomes an equality: all its “budget” w_i is used up.

Lemma 11. *Fairness Lemma For **any** vertex cover S (not just ours) and **any** fair prices p_e :*

$$\sum_{e \in E} p_e \leq w(S) = \sum_{i \in S} w_i$$

***Why?** Every edge in E is covered by at least one vertex in S . So when we sum up $\sum_{i \in S} w_i$, each edge price p_e is counted at least once. By fairness, $\sum_{e \ni i} p_e \leq w_i$ for each $i \in S$, so the total sum of prices is at most $w(S)$.*

This lemma is powerful because it gives a lower bound on any cover's cost: no matter how clever you are, the cover weight is at least $\sum_e p_e$.

8.4.3 The Pricing Algorithm

The idea is to increase edge prices gradually until we cannot do so without violating fairness, then collect all tight vertices.

Set all $p_e = 0$

$S = \text{empty set}$

While there exists an uncovered edge (i, j) :

 Pick any uncovered edge $e = (i, j)$

 Raise p_e until vertex i or vertex j becomes tight

 (i.e., until sum of prices around i equals w_i , or around j equals w_j)

$S = \{ \text{all tight vertices} \}$

Return S

Pricing Method Step by Step Graph with vertices a (weight 4), b (weight 2), c (weight 6), edge $a-b$ and edge $b-c$.

- Edge $a-b$ is uncovered. Raise p_{ab} from 0.
 b becomes tight when $p_{ab} = 2$ (since $w_b = 2$). So set $p_{ab} = 2$.
- Now vertex b is tight and is in S .
- Edge $b-c$ is covered by b (which is now in S). No more uncovered edges.
- $S = \{b\}$, but wait — a is not in S , so edge $a-b$ is covered only by b . Check: $b \in S$ covers $a-b$, and $b \in S$ covers $b-c$. ✓
- Total weight $w(S) = w_b = 2$.
- Total prices $\sum_e p_e = p_{ab} = 2$.
- $w(S) = 2 \leq 2 \cdot 2 = 4 \leq 2w(S^*)$. ✓

Theorem 35. *Pricing Method is a 2-Approximation Let S be the set returned by the pricing algorithm and S^* be any optimal cover. Then:*

$$w(S) \leq 2w(S^*)$$

Proof:

Step 1 — S is a valid vertex cover.

Every tight vertex is in S . If some edge (i, j) were not covered, then neither i nor j would be tight. But then the while loop would keep running and raise $p_{(i,j)}$ until one becomes tight. Contradiction: the loop would not have stopped. So all edges are covered.

Step 2 — The approximation bound.

Every vertex in S is tight, so $w_i = \sum_{e \ni i} p_e$ for all $i \in S$. Therefore:

$$w(S) = \sum_{i \in S} w_i = \sum_{i \in S} \sum_{e \ni i} p_e$$

Now extend the sum to all vertices in V (this only makes it larger, since prices are ≥ 0):

$$\sum_{i \in S} \sum_{e \ni i} p_e \leq \sum_{i \in V} \sum_{e \ni i} p_e = 2 \sum_{e \in E} p_e$$

The last equality holds because each edge (i, j) appears **exactly twice** in the double sum: once when i is the vertex, and once when j is.

By the Fairness Lemma, $\sum_{e \in E} p_e \leq w(S^*)$. Combining:

$$w(S) \leq 2 \sum_{e \in E} p_e \leq 2 w(S^*) \quad \square$$

8.5 Coping with NP-completeness

When we face an NP-hard problem, we cannot have all three of these perfect features:

1. Solve arbitrary (any) instances of the problem.
2. Solve the problem to optimality (find the exact best solution).
3. Solve the problem in polynomial time.

Since we must sacrifice one, we choose to sacrifice **optimality**.

Definition 36. ρ -Approximation Algorithm An algorithm is a ρ -approximation algorithm if it is guaranteed to:

- Run in polynomial time.
- Solve arbitrary instances of the problem.
- Find a solution whose value is within a ratio ρ of the true optimal value.

The Challenge: How do we prove our solution is close to the optimum, when we don't even know what the exact optimal value is?

8.6 Load Balancing (Section 11.1)

8.6.1 The Problem

Input: m identical machines and n jobs. Job j has a processing time t_j .

Rules: A job must run continuously on one machine. A machine processes one job at a time.

Load: Let $J(i)$ be the jobs assigned to machine i . The load of machine i is $L_i = \sum_{j \in J(i)} t_j$.

Makespan: The maximum load on any machine: $L = \max_i L_i$.

Goal: Assign jobs to machines to **minimize the makespan** L .

Note: Load balancing on just 2 machines is already NP-hard (it reduces from the NUMBER-PARTITIONING problem).

8.6.2 Greedy Algorithm: List Scheduling

Algorithm (Graham, 1966):

- Consider the n jobs in any fixed order.
- Assign job j to the machine whose load is **smallest so far**.

8.6.3 Analysis of List Scheduling

To prove how good the algorithm is, we need lower bounds on the optimal makespan L^* .

Lemma 12. *Lower Bounds on L^**

1. **Max job:** $L^* \geq \max_j t_j$. (Some machine must process the longest job).
2. **Average load:** $L^* \geq \frac{1}{m} \sum_j t_j$. (The total work is shared among m machines, so at least one machine must do $1/m$ of the total work).

Theorem 36. *List Scheduling is a 2-approximation. The greedy list scheduling algorithm produces a makespan $L \leq 2L^*$.*

Proof: Let machine i be the bottleneck machine (the one that achieves the max load $L_i = L$).

Let job j be the **last job** scheduled on machine i .

When job j was assigned to i , machine i had the smallest load. So its load before getting j was $L_i - t_j$.

This means $L_i - t_j \leq L_k$ for all other machines k .

Summing these inequalities over all m machines and dividing by m :

$$L_i - t_j \leq \frac{1}{m} \sum_{k=1}^m L_k = \frac{1}{m} \sum_k t_k \leq L^* \quad (\text{by Lemma 2})$$

Now, we calculate the total load of the bottleneck machine:

$$L = L_i = (L_i - t_j) + t_j \leq L^* + L^* = 2L^* \quad (\text{since } t_j \leq L^* \text{ by Lemma 1})$$

Thus, $L \leq 2L^*$. $\square$

Is the 2-approximation tight? Yes. Consider m machines, with $m(m-1)$ jobs of length 1, followed by one job of length m .

The greedy algorithm will distribute the short jobs evenly (load $m-1$ on all machines), and then put the big job (length m) on the first machine. Makespan = $2m-1$.

Optimal solution: put the big job on its own machine, and distribute the small jobs on the other $m-1$ machines. Optimal makespan = m .

Ratio $\approx \frac{2m-1}{m} \approx 2$.

8.6.4 Longest Processing Time (LPT) Rule

To improve the result, we can sort the jobs first!

LPT Rule: Sort the n jobs in **descending order** of processing time ($t_1 \geq t_2 \geq \dots \geq t_n$), then run the list scheduling algorithm.

Lemma 13. *LPT Lower Bound* If there are more than m jobs, then $L^* \geq 2t_{m+1}$.

Proof: Consider the first $m+1$ jobs. They are the largest. By the pigeonhole principle, at least one machine must get two of these jobs. The smallest of these jobs is t_{m+1} , so that machine has load at least $2t_{m+1}$.

Theorem 37. *LPT is a $3/2$ -approximation Using LPT, the makespan is $L \leq \frac{3}{2}L^*$. (Graham actually proved it is a $4/3$ -approximation).*

Proof sketch: Following the same logic as before: $L = (L_i - t_j) + t_j \leq L^* + t_j$. Since jobs are sorted and we assume $n > m$, the last job $j \geq m + 1$, so $t_j \leq t_{m+1} \leq \frac{1}{2}L^*$ (from the Lemma).

Therefore, $L \leq L^* + \frac{1}{2}L^* = \frac{3}{2}L^*$. $\square$

8.7 Center Selection (Section 11.2)

8.7.1 The Problem

Input: A set of n sites $s_1, \dots, s_n$ and an integer $k > 0$.

Distances: We have a distance function between sites satisfying:

- Identity: $\text{dist}(x, x) = 0$
- Symmetry: $\text{dist}(x, y) = \text{dist}(y, x)$
- Triangle inequality: $\text{dist}(x, y) \leq \text{dist}(x, z) + \text{dist}(z, y)$

Goal: Select a set C of k centers to minimize the **maximum distance** from any site to its nearest center. This maximum distance is the covering radius: $r(C) = \max_i \text{dist}(s_i, C)$.

8.7.2 Greedy Algorithm: A False Start

Bad idea: Put the first center at the absolute best location for a single center, then keep adding centers to reduce the radius as much as possible.

Result: This can be arbitrarily bad!

8.7.3 The Correct Greedy Algorithm

Algorithm: Repeatedly choose the next center to be the site **farthest from any existing center**.

```
C = {s_1} // Pick an arbitrary first site
Repeat k-1 times:
    Select a site s_i that maximizes dist(s_i, C)
    C = C U {s_i}
Return C
```

Property: When the algorithm finishes, all centers in C are separated from each other by at least $r(C)$.

Theorem 38. *Center Selection 2-Approximation Let C^* be the optimal set of centers. Then $r(C) \leq 2r(C^*)$.*

Proof by contradiction:

Assume our radius is strictly larger than twice the optimal: $r(C^*) < \frac{1}{2}r(C)$.

For each of our chosen centers $c_i \in C$, draw a ball of radius $\frac{1}{2}r(C)$.

Because all centers in C are at least $r(C)$ apart, these balls do not overlap.

Since we have k non-overlapping balls, the optimal solution C^* (which also has k centers)

must place exactly one center c_i^* in each ball.

Now take any site s . Let c_i^* be its closest optimal center. By the triangle inequality:

$$\text{dist}(s, C) \leq \text{dist}(s, c_i) \leq \text{dist}(s, c_i^*) + \text{dist}(c_i^*, c_i) \leq r(C^*) + r(C^*) \leq 2r(C^*)$$

So $r(C) \leq 2r(C^*)$, which contradicts our assumption! $\square$

Our algorithm only places centers **on the sites themselves**, but it is still within a factor of 2 of the optimal solution that is allowed to place centers **anywhere**.

Can we do better (e.g. 1.5-approximation)?

No. Unless $P = NP$, there is no ρ -approximation for $\rho < 2$. Doing so would allow us to solve the NP-complete DOMINATING-SET problem in polynomial time.

8.8 Pricing Method: Vertex Cover (Section 11.4)

8.8.1 Weighted Vertex Cover

Definition: Given a graph $G = (V, E)$ where each vertex i has a weight $w_i \geq 0$, find a vertex cover $S \subseteq V$ (a set of vertices that covers every edge) of **minimum total weight**.

8.8.2 The Pricing Method Concept

Imagine each edge $e = (i, j)$ must pay a “price” $p_e \geq 0$ to use the vertices i and j .

Fairness rule: The total price paid by all edges connected to a vertex i cannot exceed the weight w_i of that vertex.

$$\sum_{e=(i,j)} p_e \leq w_i$$

Lemma 14. Fairness Lemma For any valid vertex cover S and any fair prices p_e , the total sum of all edge prices is less than or equal to the weight of the cover:

$$\sum_{e \in E} p_e \leq w(S)$$

Proof: Each edge is covered by at least one node in S . If we sum the fairness inequalities for all nodes in S , we get an upper bound on the sum of all prices.

8.8.3 Pricing Method Algorithm

We set prices and find the vertex cover at the same time:

S = empty set

For all edges, set $p_e = 0$

While there is an edge (i, j) where NEITHER i nor j is "tight":

Select that edge $e = (i, j)$

Increase p_e as much as possible until either i or j becomes "tight"

(A vertex is "tight" when the sum of prices of its edges exactly equals its weight)

S = the set of ALL tight nodes

Return S

Theorem 39. *Pricing Method is a 2-approximation The pricing method returns a vertex cover S such that $w(S) \leq 2w(S^*)$, where S^* is the optimal vertex cover.*

Proof:

1. **It is a valid vertex cover:** If an edge is uncovered, neither of its endpoints is tight, which means the *while* loop wouldn't have terminated.
2. **Approximation ratio:** Because every node in our set S is tight:

$$w(S) = \sum_{i \in S} w_i = \sum_{i \in S} \sum_{e=(i,j)} p_e$$

Since $S \subseteq V$, we can sum over all nodes in V to get an upper bound. In that sum, each edge $e \in E$ is counted **exactly twice** (once for each of its two endpoints):

$$\sum_{i \in S} \sum_e p_e \leq \sum_{i \in V} \sum_e p_e = 2 \sum_{e \in E} p_e$$

By the Fairness Lemma, the sum of all prices is $\leq w(S^*)$. Therefore, $w(S) \leq 2w(S^*)$. $\square$